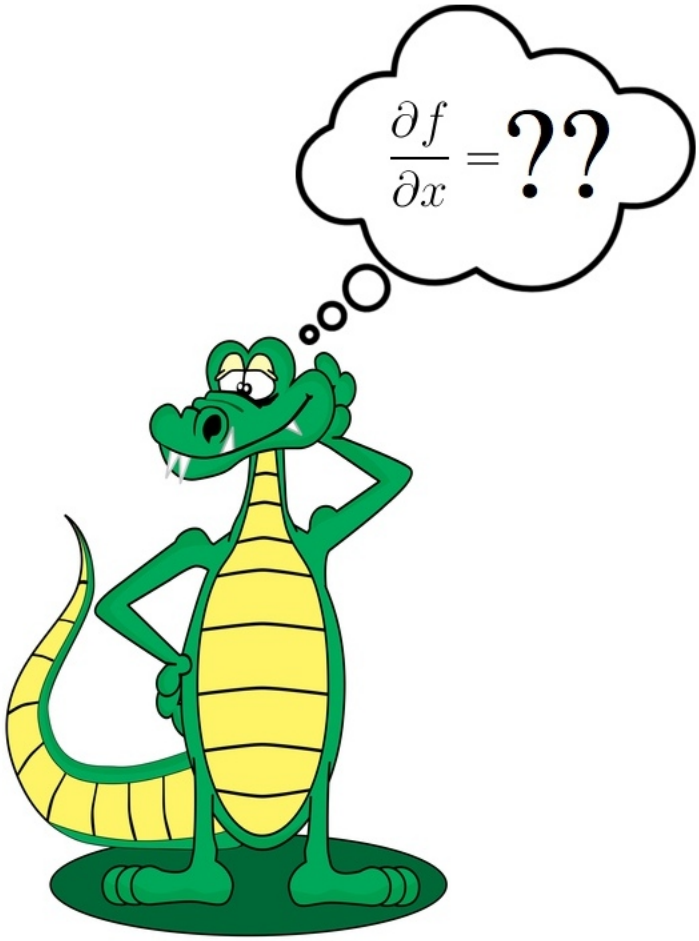

ADiGator

A Source Transformation via Operator Overloading
Toolbox for the Automatic Differentiation of
Mathematical Functions in MATLAB

Matthew J. Weinstein
Anil V. Rao

Embedded fork — v2.0

Pedro Lourenço (GMV)

Contents

1 Introduction

ADiGator is a MATLAB automatic differentiation package which transforms user function files into derivative function files. These derivative function files are written completely in terms of the native MATLAB commands and thus may be evaluated on numeric input values to calculate the numeric derivative of the output with respect to a defined variable of differentiation. These numeric evaluations may be performed as many times as desired without generating a new derivative file, as long as the input sizes and derivative sparsity patterns are not to change. In the event that a user wishes to obtain derivatives of the same function file, but evaluated on different input sizes and/or derivative sparsity patterns, then a new derivative file must be generated. The package is particularly appealing for applications where the same derivative must be found at multiple different points, i.e. non-linear root finding/optimization, stiff ode integration, etc. For details on the methodology behind the algorithm and the overloaded class which is used, the user is referred to [1], and [2], which can be seen here and here, respectively. In the following user's guide we will explain how to use the *ADiGator* package. For more information on Automatic Differentiation in general, please refer to [3]. For an explanation on the mathematical notations used within this guide, please see the Appendix.

This document describes the base *ADiGator* package. The embedded fork (v2.0; GMV / Pedro Lourenço, 2025–2026) adds derivative files suitable for embedded code generation (MATLAB Coder), together with several capability and output-form extensions. These are documented in Section 6 and as additional options in Table 1; the underlying differentiation algorithm is unchanged.

2 Installation

The *ADiGator* package is written entirely in MATLAB and thus should be usable on any operating system as long as MATLAB is installed. The code has been tested in MATLAB versions 7.11 through 8.2. To install the package, one needs to unpack the zip file and add two directories to the MATLAB path. This may be done as follows:

1. Unpack the zip file into a convenient location - ensure that the folder permissions are set to read and write.
2. Within MATLAB, change the current directory to the location which the zip file was unpacked.
3. Run the file `startupadigator`, you can do this by typing `startupadigator` in the MATLAB command window.
4. (optional) If you wish to not do this each time you restart MATLAB, then type `savepath` in the MATLAB command window.
5. (optional) To run a set of example problems, change to the 'examples' directory and type 'runAllExamples'.

3 Using the adigator Function to Generate Derivative Files

In order to obtain a numeric derivative the user must first generate a derivative file using the *ADiGator* package and may then evaluate the generated file numerically to obtain a derivative. It is stressed that after the file is generated, the software is no longer being used. Furthermore, it is noted that the derivative files are generated for a fixed input size to the user's function file. Thus, as long as the input sizes do not change, the same derivative file may be used to evaluate the derivative at multiple points. We will now explain in detail the commands required to transform a user function file into a derivative function file and then explain how to evaluate the generated derivative file. We will then demonstrate both the generation and evaluation of a derivative file using a simple example.

3.1 Generating Derivative Files

The derivative files may be generated in a three step process, where first the user must write the function file to be differentiated, second the user must define the inputs to the function file and identify which inputs contain derivative information, and then finally the transformation is initialized by using the `adigator` command. In this section we present these three steps.

3.1.1 The User Function File

In order to generate derivative files, we first need a function file of which we are to differentiate. This user written function file should be written such that one or more of the inputs is to have derivatives with respect to some *variable of differentiation* (VOD), where it is desired to determine the derivatives of the outputs of the file with respect to the same variable of differentiation. The primary restrictions placed on the input/output scheme of these functions is that 1. there is at least one input/output and 2. one of the inputs (or a field of an input structure or an element of an input cell array) has derivatives with respect to the VOD. For more user function file coding restrictions please refer to Section 7.

3.1.2 Identifying the User Function Inputs

Prior to calling the main transformation routine, `adigator`, the user must define the inputs to the function which they just created. Here we define three different types of inputs and explain how they are to be identified:

- **Derivative Inputs**: This refers to any input arrays which are to have derivatives with respect to the VOD; these inputs are generated with the `adigatorCreateDerivInput` command, whose syntax is as follows:

  ```
  x = adigatorCreateDerivInput(xsize,derivinfo)
  ```

  where

  - `xsize` $= [m, n]$ is the size of the input array
  - `derivinfo` gives information on the derivatives, this may be defined in one of two ways:

    1. If the input being created is the variable of differentiation (and thus has a derivative equal to the $mn \times mn$ identity matrix) then `derivinfo` can simply be set to a string name which you wish to call the VOD.
    2. Otherwise, `derivinfo` must be a structure array defining the VOD to which the input has derivatives with respect to, as well as the possible non-zero locations of the *unrolled* Jacobian of the input with respect to the VOD. These fields must be:
       - `derivinfo.vodname` = string name which uniquely defines the VOD
       - `derivinfo.vodsize` $= [p, q]$, the size of the VOD
       - `derivinfo.nzlocs` $= [\mathbf{i}, \mathbf{j}]$, where $\mathbf{i}, \mathbf{j} \in \mathbb{Z}_+^{nz}$ define the row and column locations of the non-zero elements of the $mn \times pq$ Jacobian of the input with respect to the VOD. Note: these should be in the natural MATLAB indexing order, that is, if the user has built the unrolled Jacobian in MATLAB and called it `Jac`, then $\mathbf{i}$ and $\mathbf{j}$ could be found by `[I,J] = find(Jac)`.

    Please see the Appendix for further clarification on the term "unrolled Jacobian".

- **Unknown Auxiliary Inputs**: This refers to any input arrays to the user function which do not contain derivatives with respect to the VOD, but are not a fixed, known, numeric value. That is, they may change numeric values on any given call to the user function file. These inputs are generated with the `adigatorCreateAuxInput` command, whose syntax is as follows:

  ```
  x = adigatorCreateAuxInput(xsize)
  ```

  ,

  where `xsize` $= [m, n]$ is the size of the input array.
  NOTE: An alternative to defining Unknown Auxiliary Inputs is discussed in Section 4 by using the `auxdata` option.

- **Known Auxiliary Inputs**: This refers to any input arrays to the user function which have a fixed, known, numeric value. These inputs should simply be assigned their fixed value.

Here we note that the above three types of inputs only refer to numeric arrays, and that, if a user function was written to take a structure input with numeric fields, then the structure should be built, and the aforementioned inputs should be assigned to the proper fields. Likewise, if an input is to be a cell array, then the cell array should be built and the aforementioned input types should be assigned to the appropriate elements of the cell array.

3.1.3 Calling the Derivative File Generation

After having created the function file to be differentiated and creating all derivative/unknown auxiliary/known auxiliary inputs, the user may now use the `adigator` command to generate the derivative file. The syntax for this call is as follows:

```
Outputs = adigator(UserFunFileName,Inputs,DerivFileName)
```

or

```
Outputs = adigator(UserFunFileName,Inputs,DerivFileName,options)
```

where

- `UserFunFileName` = string name of the user function to be differentiated
- `Inputs` = $1 \times N$ cell array, where N denotes the number of inputs to the user function and element i contains input i to the user's function file
- `DerivFileName` = string name which the derivative file is to be called
- `Outputs` = $1 \times M$ cell array, where M denotes the number of outputs of the user's function file. Cell i will contain the size and possible non-zero derivative locations of the i^{th} output.
- `options`(optional) = option structure which can be generated using the `adigatorOptions` function as defined in Section 4.

Using this command will then generate the derivative file, `DerivFileName.m`, as well as a MATLAB binary file, `DerivFileName.mat`, where `DerivFilename` is as the user specified.

3.2 Evaluating the Generated Derivative File

As previously stated, after the derivative files have been generated, the *ADiGator* software is no longer needed, but rather only the generated `.m` and `.mat` files are needed to compute the numeric derivative.[1] While the input/output structure of the generated derivative function file is similar to the input/output structure of the user defined function file, it is not exactly the same. We now look at how both the input and output structures change.

3.2.1 Derivative File Input

The only difference between the user function file input scheme and the generated derivative file input scheme is that, for any **Derivative Inputs**, these inputs must now be given as structures with a *function field* and *derivative field*. Suppose that a **Derivative Input** was defined as a $m \times n$ array with nz possible non-zero derivatives with respect to a VOD which was given the name `'vod'`. Then the input to the derivative file for this variable would have the following two fields:

- *function field*: this field will always be given the name `.f` and must be assigned the $m \times n$ numeric array corresponding to the function values.

[1] The only exception to this is when differentiating a file which calls `interp2`. In this case, the file will also depend upon the ADiGator function `adigatorEvalInterp2pp`.

- *derivative field*: this field will be given a name corresponding to the defined VOD name, if the VOD name was given as `'vod'`, then the field would be `.dvod`, if the VOD name was given as `'x'`, then the field would be `.dx`, etc. Furthermore, this field must be assigned a column vector of length nz corresponding to the possible non-zero derivatives of the input with respect to the VOD.

Any inputs defined as **Unknown Auxiliary Inputs** may be assigned any numeric values, as long as the array is the same size as was given to the `adigatorCreateAuxInput` command, and any inputs defined as **Known Auxiliary Inputs** should be assigned the values which were assigned prior to the call to `adigator`.

3.2.2 Derivative File Output

Similar to any **Derivative Inputs**, all outputs of the derivative file which correspond to numeric array outputs of the function file will now be structures with different fields. Each will contain a *function field*, `.f` containing the values of the function, and if the output contains derivatives with respect to the VOD, then it will also be assigned the following three fields:

- *derivative value field*: (for example `.dx` if the VOD name is `'x'`) This will contain a column vector of length nz corresponding to the possible non-zero elements of the output with respect to the VOD.
- *derivative size field*: (for example `.dx_size` if the VOD name is `'x'`) This will contain the size of the derivative matrix, for example, if the output variable is a vector of length m and the VOD is a vector of length n, then this will be assigned the value $[m, n]$.
- *derivative location field*: (for example `.dx_location` if the VOD name is `'x'`) This will contain a $nz \times d$ integer array of indices which map the values stored in the *derivative value field* into an array of the size stored in the *derivative size field*. Here we note that d is equal to the length of the size stored in the derivative size field, and that column i ($i = 1, \ldots, d$) of the location field gives the locations for the i^{th} dimension of the derivative array.

3.3 Illustrative Example

For this example, assume we have written some function, `y = myfun(x,k,N)`, where we wish to take the derivative of $\mathbf{y} \in \mathbb{R}^4$ with respect to the input $\mathbf{x} \in \mathbb{R}^3$, and that the input $\mathbf{k} \in \mathbb{R}^3$ is a vector of unknown auxiliary values and the input $N = 3$ is a fixed value. Prior to generating the derivative file, we would first need to identify our inputs. Supposing we wish to give our VOD the name `'x'`, we could define our **Derivative Input** in one of two ways. The first, simpler way would be:

```
x = adigatorCreateDerivInput([3 1],'x');
```

or, to the same end, we could do

```
derivinfo.vodname = 'x';
derivinfo.vodsize = [3 1];
derivinfo.nzlocs  = [1 1; 2 2; 3 3];
x = adigatorCreateDerivInput([3 1],derivinfo);
```

Next, we need to define our inputs for $\mathbf{k}$ and N, so we would do this as

```
k = adigatorCreateAuxInput([3 1]);
N = 3;
```

We now have our inputs defined and may generate the derivative code. Supposing we wished to name the derivative file `'myderiv'`, we would call `adigator` as

```
adigator('myfun',{x,k,N},'myderiv');
```

This would then generate the files `myderiv.m` and `myderiv.mat`. In order to now evaluate the derivative we need to redefine our inputs for $\mathbf{x}$ and $\mathbf{k}$. We would do this as follows:

```
x = struct('f',rand(3,1),'dx',ones(3,1));
k = rand(3,1);
```

where we are assigning random values to $\mathbf{x}$ and $\mathbf{k}$. Furthermore, we note that the derivative of $\mathbf{x}$ with respect to $\mathbf{x}$ is the 3×3 identity matrix, and that the non-zero elements are given by a vector of ones (the value assigned to `x.dx`). We could then evaluate the derivative file by

```
y = myderiv(x,k,N);
```

then `y.f` would be a vector of length 4. Supposing that the derivative of $\mathbf{y}$ has the sparsity pattern given by

$$\text{struct}(\mathbf{Jy}(\mathbf{x})) = \begin{array}{c} \\ y_1 \\ y_2 \\ y_3 \\ y_4 \end{array} \begin{array}{c} \begin{array}{ccc} x_1 & x_2 & x_3 \end{array} \\ \begin{bmatrix} \bullet & \bullet & \bullet \\ & \bullet & \\ & & \bullet \\ \bullet & \bullet & \end{bmatrix} \end{array} \tag{1}$$

then `y.dx` would be a column vector of length 7, `y.dx_size` would be assigned `[4, 3]` and `y.dx_location` would be `[1 1;4 1;1 2;2 2;4 2;1 3;3 3]`. Furthermore, if we wished to build a sparse MATLAB array corresponding to the Jacobian, $\mathbf{Jy}(\mathbf{x})$, we could do so using the MATLAB `sparse` command as follows:

```
Jac = sparse(y.dx_location(:,1),y.dx_location(:,2),y.dx,y.dx_size(1),y.dx_size(2));
```

If we wished to build a non-sparse MATLAB array corresponding to the Jacobian, $\mathbf{Jy}(\mathbf{x})$, we could do so using a linear index as follows:

```
Jac = zeros(y.dx_size);
index = sub2ind(y.dx_size,y.dx_location(:,1),y.dx_location(:,2));
Jac(index) = y.dx;
```

Here it is stressed that this file may be evaluated as many times as desired using different values assigned to `x.f` and `k`, which will produce different values of `y.f` and `y.dx`, but `y.dx_size` and `y.dx_location` will always be the same.

4 ADiGator Options

The *ADiGator* package currently has different options which may given to the `adigator` command. In order to build the options structure use the following command:

```
options = adigatorOptions(field1,value1,field2,value2,...)
```

A list of all options, values, and descriptions is given in Table 1.

Table 1: Options for ADiGator

Option Field	Value	Description
AUXDATA	1	**Known Auxiliary Inputs** (as defined in Section 3.1.2) will always have the same sparsity patterns as given to `adigator`, but their non-zero values may change.
	0	**Known Auxiliary Inputs** will always have the same numeric values (default)
ECHO	1	Echo to MATLAB command screen during the transformation process (default)
	0	Do not echo to MATLAB command screen
UNROLL	1	When this is set, any loops and/or sub-functions encountered will be unrolled in the generated derivative file
	0	keep loops and sub-functions rolled in the derivative file (default)

continued on next page

Table 1 – continued from previous page

Option Field	Value	Description
COMMENTS	1	Print comments to the derivative file giving the lines of user code which correspond to printed derivative statements (default)
	0	Do not print the comments.
OVERWRITE	1	If a `DerivFileName` is given to `adigator` such that a file already exists with the given name, setting this option will automatically overwrite the file.
	0	If this option is set, `adigator` will error out rather than overwrite the derivative file. (default)
MAXWHILEITER	$k > 0$	For use with `while` loops. The maximum number of iterations, k, that the algorithm will attempt to find a static loop iteration during file generation. (default is $k = 10$).
COMPLEX	0	Binary flag for turning on and off complex variable handling. If set to 1, the tool will expect complex variables and thus use the complex forms of ctranspose, abs, dot, etc. Otherwise, it will assume no complex numbers exist in the user's program.
EMBED_MODE	`[]`	**(new in v2.0)** Unset (default): each entry point resolves its own default — `adigatorGenDerFile_embedded` uses `'i'` (inline), the other generators use `'c'` (classic). Set explicitly to override.
	`'c'`	**C**lassic. The static index/data tables are stored in a `.mat` file, loaded at runtime into a global variable. Not suitable for embedded code generation.
	`'i'`	**I**nline. The static tables are emitted as source in a companion data function: no `.mat`, no global/persistent variables, and no runtime load. Fully self-contained for code generation.
PATH	`[]`	**(new in v2.0)** Directory in which to place all generated files. If empty, the calling directory is used (default).
LOOPBOUND	`{}`	**(new in v2.0)** All loop bounds are fixed at their generation-time trip counts (default).
	name(s)	Char/string/cellstr naming input(s) of the differentiated function which act as runtime loop bounds: the file is generated at the maximum trip count (the value passed to `adigator`) and may then be evaluated for any `n` up to that maximum. See Section 6.4.
DER_OUTPUT	`'matrix'`	**(new in v2.0)** The canonical output-form option: the wrapper projects the *top-order* derivative into a dense/sparse matrix on every call (default).
	`'csc'`	The wrapper returns the top-order derivative's structurally-possible-nonzero vector in compressed-sparse-column (CSC) order (values only — no dense projection or scatter); the constant pattern is exported once as the per-role `output.*CSC` companion (`JacobianCSC`, `GradientCSC`, `HessianCSC`; fields `Size`/`ColPtr`/`RowIdx`/`Nnz`/`IndexBase`), the sole exported sparse-pattern representation in both modes. On a Hessian file this flips only the Hessian output — the gradient companion `Grd` stays dense.
DER_LEVELS	`[]`	**(new in v2.0)** The wrapper returns every derivative level its generator produces (default).
	vector	A subset of $\{0, 1, 2\}$ (0 = function value, 1 = first derivative, 2 = Hessian) selecting which levels the wrapper returns; the top level the generator is named for is always included.

continued on next page

Table 1 – continued from previous page

Option Field	Value	Description
SLIM_EMBED	[]	**(new in v2.0)** Unset (default): `adigatorGenDerFile_embedded` defaults this on (embeddable output is wanted); the other generators leave it off. Set 0/1 to override.
	0	No slimming.
	1	In inline mode (`'i'`), slice from the generated code the statements that feed only output fields the wrapper never reads (the `_location`/`_size` metadata), so their constant index tables drop from the embedded data. Guarded by a generation-time correctness check; left unsliced on any uncertainty.

5 Generating Derivative Files and Wrapper Files via Black Box ADiGator Commands

As seen in Section 3, the call to the `adigator` routine can be somewhat complicated. Moreover, the derivative files which are generated have a very distinct input/output structure. The complexity of the `adigator` command together with the input/output structure of the generated files allows one to differentiate files which have complex input/output structures and to specifically define their problem. That being said, it is often the case that the user simply wants to generate a Jacobian file, Hessian file, etc. and is forced to create wrapper files for the `adigator` generated files. For this reason, some "black box" file generation routines have been added. These routines simply take information from the user, call the `adigator` routine one or more times, and then writes a wrapper file(s) with a basic input/output structure. In this section, the various black box routines are described.

5.1 adigatorGenJacFile

When to use: You wish to compute the Jacobian of a function with respect to one of its inputs.

User Function Input Restrictions: One of the inputs must be the variable of differentiation (cannot be embedded within a cell/structure). The file may only have a single output.

Generated Jacobian File Info: The Jacobian wrapper file which is generated has the name of the user's function with `_Jac` appended. It takes the same inputs as the original file, but returns both the Jacobian and original function value. The Jacobian is always computed and returned as the first output.

Usage	`output = adigatorGenJacFile(UserFunName,UserFunInputs)`
Input Info	`UserFunName` - string name of file to be differentiated `UserFunInputs` - cell array of inputs to user's function The cell element of `UserFunInputs` corresponding to the variable of differentiation must be created via `adigatorCreateDerivInput`. All other inputs must be fixed.

Output Info	`output.FunctionFile` - same as `UserFunName`, ex: `'myfun'`
	`output.JacobianFile` - string name of generated Jacobian file, ex: `'myfun_Jac'`
	`output.JacobianCSC` - the CSC sparsity pattern (`Size`/`ColPtr`/`RowIdx`/`Nnz`/`IndexBase`) of the Jacobian

Examples	`examples/jacobians/arrowhead/`
	`examples/jacobians/polydatafit/`
	`examples/stiffodes/brusselator/`

5.2 adigatorGenHesFile

When to use: You wish to compute the Hessian of a function with respect to one of its inputs.

User Function Input Restrictions: One of the inputs must be the variable of differentiation (cannot be embedded within a cell/structure). The file may only have a single output.

Generated Gradient File Info: The Gradient wrapper file which is generated has the name of the user's function with `_Grd` appended. It takes the same inputs as the original file, but returns both the Gradient and original function value. The Gradient is always computed and returned as the first output.

Generated Hessian File Info: The Hessian wrapper file which is generated has the name of the user's function with `_Hes` appended. It takes the same inputs as the original file, but returns the Hessian together with the Gradient and original function value. The Hessian is always computed and returned as the first output, followed by the gradient and function.

Usage	`output = adigatorGenHesFile(UserFunName,UserFunInputs)`
Input Info	`UserFunName` - string name of file to be differentiated `UserFunInputs` - cell array of inputs to user's function The cell element of `UserFunInputs` corresponding to the variable of differentiation must be created via `adigatorCreateDerivInput`. All other inputs must be fixed.

Output Info	`output.FunctionFile` - same as UserFunName, ex: `'myfun'`
	`output.GradientFile` - string name of generated Gradient file, ex: `'myfun_Grd'`
	`output.HessianFile` - string name of generated Gradient file, ex: `'myfun_Hes'`
	`output.HessianCSC` - the CSC sparsity pattern of the Hessian (and `output.GradientCSC` for the compa

Examples	`examples/hessians/logsumexp/`

5.3 Solver-Integration Wrappers (Inherited from Upstream)

Note — deprecated. The five `adigatorGenFiles4*` commands documented in the following subsections (for `fsolve`, `fminunc`, `fmincon`, IPOPT, and GPOPS II) are convenience wrappers carried over from upstream *ADiGator*. They are **deprecated**: shipped for **continuity with the upstream repository only**, **not maintained**, and their functionality **may be reduced or removed in a future release** without further notice. They emit **host-only** derivative files (the classic runtime-`load`/global mechanism) and are **not at feature parity** with the rest of this fork: they do not support embedded code generation (`EMBED_MODE`, MATLAB / Embedded Coder), the `PATH` option, the `csc` output form, or reverse mode, and are not maintained or tested to the depth of the core generators. For embeddable derivatives use the core generators instead (Section 6); these wrappers are retained only for upstream continuity, and new code should not be written against them.

5.4 adigatorGenFiles4Fsolve

When to use: You wish to compute the Jacobian of a function with respect to its first input to be used with `fsolve`. (`fsolve` is provided in the MATLAB optimization package and is used for solving a system of non-linear equations)

User Function Input Restrictions: The first input must be the variable of differentiation. One extra variable is allowed for auxiliary data if desired.

Generated Jacobian File Info: The Jacobian wrapper file which is generated has the name of the user's function with `_Jac` appended. It takes the same inputs as the original file. If only one output is requested, the file only computes the original function. If two outputs are requested, the file computes the Jacobian and original function.

Usage	`funcs = adigatorFiles4Fmincon(setup)`
Input Info	`setup.function` - string name of file to be differentiated `setup.numvar` - length of the variable of differentiation `setup.auxdata` - (optional) if the user function has an auxdata input, then it should be included here
Output Info	`funcs.jacobian` - Jacobian file function handle - if `auxdata` specified, it is fixed in this handle.
Examples	`examples/optimization/fsolveEx/`

5.5 adigatorGenFiles4Fminunc

When to use: You wish to compute the gradient and/or Hessian of an objective function with respect its first input and supply gradient/Hessian to `fminunc`. (`fminunc` is provided in the MATLAB optimization package and is used for solving unconstrained optimization problems)

User Function Input Restrictions: The first input must be the variable of differentiation. One extra variable is allowed for auxiliary data if desired.

Generated Gradient File Info: The gradient wrapper file which is generated has the name of the user's function with `_Grd` appended. It takes the same inputs as the original file. If only one output is requested, the file only computes the objective. If two outputs are requested, the file computes the gradient and objective.

Generated Hessian File Info: The Hessian wrapper file which is generated has the name of the user's function with `_Hes` appended. It takes the same inputs as the original file. If only one output is requested, the file only computes the original function. If two outputs are requested, the file computes the gradient and objective. If three outputs are requested, the file computes the Hessian, gradient, and objective.

Usage	`funcs = adigatorGenFiles4Fminunc(setup)`
Input Info	`setup.objective` - string name of user's objective function `setup.numvar` - length of the variable of differentiation `setup.order` - 1 or 2 `setup.auxdata` - (optional) if the user function has an auxdata input, then it should be included here
Output Info	`funcs.gradient` - gradient file function handle (if `setup.order` = 1) `funcs.hessian` - Hessian file function handle (if `setup.order` = 2) If `auxdata` specified, it is fixed in the function handle. The gradient file will not be generated if `setup.order`=2.
Examples	`examples/optimization/fminuncEx/`

5.6 adigatorGenFiles4Fmincon

When to use: You wish to compute an objective gradient/constraint Jacobian/Lagrangian Hessian for use with `fmincon`. (`fmincon` is provided in the MATLAB optimization package and is used for solving constrained optimization problems)

User Function Input Restrictions: The first input must be the variable of differentiation. One extra variable is allowed for auxiliary data if desired. If auxiliary data is used in objective function, same auxiliary data structure should be passed to constraint function (and vice verse).

Generated Objective Gradient File Info: The gradient wrapper file which is generated has the name of the user's objective function with `_Grd` appended. It takes the same inputs as the original file. If only one output is requested, the file only computes the objective. If two outputs are requested, the file computes the gradient and objective.

Generated Constraint Gradient File Info: The gradient wrapper file which is generated has the name of the user's constraint function with `_Grd` appended. It takes the same inputs as the original file. If only two outputs are requested, the file only computes the constraints. If four outputs are requested, the file computes the constraint gradients and constraints.

Generated Lagrangian Hessian File Info: The Hessian wrapper file which is generated has the name of the user's objective function with `_Hes` appended. It takes the same inputs as the original file, with the addition of a variable of NLP multipliers. The file always computes and returns just the Lagrangian Hessian.

Usage	`funcs = adigatorGenFiles4Fmincon(setup)`
Input Info	`setup.objective` - string name of user's objective function `setup.constraint` - (optional) string name of user's constraint function `setup.numvar` - length of the variable of differentiation `setup.order` - 1 or 2 `setup.auxdata` - (optional) if the objective/constraint function has an auxdata input, then it should be included here
Output Info	`funcs.objgrad` - objective gradient function handle `funcs.consgrad` - constraint gradient function handle (if `setup.constraint`) specified) `funcs.hessian` - Lagrangian Hessian file function handle (if `setup.order` = 2) If `auxdata` specified, it is fixed in all function handles.
Examples	`examples/optimization/fminconEx/`

5.7 adigatorGenFiles4Ipopt

When to use: You wish to compute an objective gradient/constraint Jacobian/Lagrangian Hessian for use with *IPOPT* MEX file. (*IPOPT* [4, 5] is an open source NLP solver, information on the MATLAB interface, including precompiled MEX files, may be found here)

User Function Input Restrictions: The first input must be the variable of differentiation. One extra variable is allowed for auxiliary data if desired. If auxiliary data is used in objective function, same auxiliary data structure should be passed to constraint function (and vice verse).

Generated Objective Gradient File Info: The gradient wrapper file which is generated has the name of the user's objective function with `_Grd` appended. It takes the same inputs as the original file and only returns the gradient.

Generated Constraint Jacobian File Info: The Jacobian wrapper file which is generated has the name of the user's constraint function with `_Jac` appended. It takes the same inputs as the original file and returns the sparse constraint Jacobian.

Generated Lagrangian Hessian File Info: The Hessian wrapper file which is generated has the name of the user's objective function with `_Hes` appended. It takes the same inputs as the original file, with the addition of two NLP multiplier variables, on for the objective, and one for the constraints. The file always computes and returns the sparse Lagrangian Hessian.

Usage	`funcs = adigatorGenFiles4Fmincon(setup)`
Input Info	`setup.objective` - string name of user's objective function `setup.constraint` - (optional) string name of user's constraint function `setup.numvar` - length of the variable of differentiation `setup.order` - 1 or 2 `setup.auxdata` - (optional) if the objective/constraint function has an auxdata input, then it should be included here
Output Info	The `funcs` structure is the required structure of function handles for the `ipopt` call. `funcs.objective` - objective function handle `funcs.gradient` - objective gradient function handle `funcs.constraints` - constraints function handle (if `setup.constraint` specified) `funcs.jacobian` - constraint Jacobian function handle (if `setup.constraint` specified) `funcs.jacobianstructure` - constraint Jacobian sparsity handle (if applicable) `funcs.hessian` - Lagrangian Hessian function handle (if `setup.order` = 2) `funcs.hessianstructure` - Lagrangian Hessian sparsity handle (if `setup.order` = 2) If `auxdata` specified, it is fixed in all function handles.
Examples	`examples/optimization/ipoptEx/`

5.8 Reverse Mode (Adjoint) Commands

The routines above accumulate derivatives in *forward* mode, whose cost scales with the number of independent variables. For a scalar-valued cost, or when only the action of the transposed Jacobian on a vector is required, *reverse* (adjoint) mode is far cheaper: it computes the full gradient (or $\mathbf{J}^\top\mathbf{v}$) in a fixed number of sweeps — one forward evaluation plus one adjoint pass — independent of the number of variables. The two black box routines below expose this mode with the same output-structure convention as the forward wrappers (the derivative first, the function value last; see `docs/DESIGN.md`).

Reverse mode is most useful for *dense* objective gradients — sums, log-sum-exp, least-squares residual norms — where the forward Jacobian would be a single full row and the adjoint gradient is one sweep; it also underlies the matrix-free transposed-Jacobian-vector product used by first-order embedded solvers. Both routines operate on the forward derivative file that `adigator` prints (a static tape of fixed-size statements with constant index maps) and emit a self-contained adjoint file, so the user code must first differentiate cleanly in forward mode with its loops *unrolled* (`adigatorOptions('unroll',1)`) and its active operations drawn from the supported set: elementary arithmetic, `mtimes`, `sum`, `reshape`, `transpose`, constant-index gathers/scatters/concatenations, and the common unary math functions (`sin`, `cos`, `exp`, `log`, `sqrt`, `tanh`, ...). Statements off the path from the variable of differentiation to the output pass through untouched; anything unsupported *on* that path raises a clear generation-time error naming the offending statement.

5.8.1 adigatorGenRevGradFile

When to use: You wish to compute the gradient of a *scalar* cost with respect to one of its inputs at a cost independent of the number of variables (reverse/adjoint mode).

User Function Input Restrictions: Exactly one input is the variable of differentiation (created via `adigatorCreateDerivInput`); the file must have a single *scalar* output. Loops must be unrolled (`adigatorOptions('unroll',1)`); the active operation set is as described above.

Generated Reverse-Gradient File Info: The wrapper file is named with the user's function name and `_RGrd` appended. It takes the same (plain numeric) inputs as the original file and returns `[Grd, Fun]` — the column gradient first, then the scalar value — computed by one forward and one reverse sweep.

Usage	`output = adigatorGenRevGradFile(UserFunName,UserFunInputs)`
Input Info	`UserFunName` - string name of file to be differentiated (single scalar output) `UserFunInputs` - cell array of inputs to user's function The cell element corresponding to the variable of differentiation must be created via `adigatorCreateDerivInput`. All other inputs must be fixed. `options` - (optional) `overwrite`, `path`, `echo`, `unroll` are honored.
Output Info	`output.FunctionFile` - name of the generated reverse-gradient file `output.MatFile` / `output.Path` - generated artifacts (`MatFile` is empty when the adjoint carries no constant data, as in the example below) Generated file signature: `[Grd,Fun] = <UserFunName>_RGrd(inputs)`
Examples	`examples/gradients/logsumexp/`

A concrete sample: the reverse-gradient file *ADiGator* generates for the weighted log-sum-exp cost $y(\mathbf{x}) = \log \sum_i e^{w_i x_i}$ (the `examples/gradients/logsumexp/` example). One forward sweep evaluates the cost; one reverse (adjoint) sweep then accumulates the entire gradient in a single pass, independent of $\dim \mathbf{x}$. Because this fully vectorised cost has a data-free adjoint, the generated file is self-contained — no `.mat`, no globals. The excerpt below is loaded directly from the committed golden fixture, so it is a slice of the same bytes the generator emits:

```
% ----------------- forward (function value) sweep ----------------- %
cada1f1 = w.*x;
cada1f2 = exp(cada1f1);
cada1f3 = sum(cada1f2);
y.f = log(cada1f3);
% ------------------------- reverse sweep -------------------------- %
cadaRGb4 = 1;
cadaRGb3 = zeros(1,1);
cadaRGb3 = cadaRGb3 + cadaRGb4./cada1f3;
cadaRGb2 = zeros(4,1);
cadaRGb2 = cadaRGb2 + cadaRGb3;
cadaRGb1 = zeros(4,1);
cadaRGb1 = cadaRGb1 + cadaRGb2.*cada1f2;
cadaRGbx = zeros(4,1);
cadaRGbx = cadaRGbx + cadaRGb1.*w;
Fun = y.f;
Grd = cadaRGbx(:);
%
```

5.8.2 adigatorGenJtVFile

When to use: You wish to compute the transposed-Jacobian-vector product $\mathbf{J}^\top \mathbf{v}$ of a vector function for a *runtime* vector `v`, without ever forming the Jacobian — the quantity gradient-based embedded solvers consume directly.

User Function Input Restrictions: As for `adigatorGenRevGradFile` (one derivative input; loops unrolled; the supported active operation set), except the user function may have a *vector* output. `v` is a runtime input the size of that output and does not participate in generation.

Generated JtV File Info: The wrapper file is named with the user's function name and `_JtV` appended. It takes the same inputs as the original file plus the runtime vector `v`, and returns `[Jtv, Fun]`, where `Jtv = J(x).'*v(:)`, in one forward plus one adjoint sweep (cost independent of `numel(x)`). A single generated file serves every `v`.

Usage	`output = adigatorGenJtVFile(UserFunName,UserFunInputs)`
Input Info	`UserFunName` - string name of file to be differentiated (vector output allowed) `UserFunInputs` - cell array of inputs to user's function; the variable of differentiation created via `adigatorCreateDerivInput`, all others fixed `options` - (optional) `overwrite`, `path`, `echo`, `unroll` are forwarded.
Output Info	`output.JtVName` - string name of the generated file, ex: `'myfun_JtV'` Generated file signature: `[Jtv,Fun] = <UserFunName>_JtV(inputs,v)`
Examples	Shares the reverse engine of `adigatorGenRevGradFile` (with the output's adjoint seeded by `v` instead of 1).

6 Embedded and Extended Features (v2.0)

The features described in this section are specific to the embedded fork of *ADiGator* (v2.0; GMV / Pedro Lourenço, 2025–2026). They extend the base package with derivative files suitable for embedded code generation (MATLAB Coder), together with several capability and output-form additions. The underlying differentiation algorithm is unchanged; for the architecture and the binding output conventions see `docs/DESIGN.md`, and for the development log see `docs/ROADMAP.md` and `docs/analyses/ANALYSIS.md`.

6.1 adigatorGenDerFile_embedded

When to use: You wish to generate an embeddable (code-generation-ready) Jacobian, gradient, or Hessian file, free of the runtime `load`/global-variable mechanism of the classic generated files.

Defaults: Because this entry point exists to produce embeddable, optimized code, an unset `EMBED_MODE` defaults to `'i'` (inline) and an unset `SLIM_EMBED` defaults to on (vs. classic / off for the other generators). Pass either option explicitly to override.

User Function Input Restrictions: One input must be the variable of differentiation (created via `adigatorCreateDerivInput`); it may be carried within a scalar struct/cell (see Section 6.3). The user function may have a single output.

Generated File Info: For `DerType 'jacobian'` the wrapper is named with `_Jac` appended; for `'gradient'`, with `_Grd`; for `'hessian'`, with `_Hes` (returning the Hessian, gradient, and function value). The wrapper takes the same inputs as the original file. The number and nature of the additional generated files depends on `EMBED_MODE`: classic `'c'` writes a wrapper, a `.mat`, and the derivative file; inline `'i'` writes a single self-contained file with no `.mat`. Both modes return numerically identical results.

Usage	`info = adigatorGenDerFile_embedded(DerType,` `UserFunName,UserFunInputs,options)`

Input Info	`DerType` - `'jacobian'`, `'gradient'`, `'gradient-reverse'`
	(reverse-mode adjoint gradient, Section 6.5), or `'hessian'`
	`UserFunName` - string name of file to be differentiated
	`UserFunInputs` - cell array of inputs to user's function; the variable of differentiation created via `adigatorCreateDerivInput`, auxiliary inputs via `adigatorCreateAuxInput`
	`options` - (optional) options structure; see `EMBED_MODE`, `PATH`, `DER_LEVELS`, `SLIM_EMBED`, `DER_OUTPUT` in Table 1

Examples	`examples/jacobians/structinput/` (struct input, inline `'i'`) `examples/hessians/logsumexp/` `examples/optimization/pipg/` (Hessian)

A minimal example, generating an inline (self-contained) Hessian of `gapfun` with respect to its second input:

```
z    = adigatorCreateDerivInput([2 1],'z');  % variable of differentiation
```

```
w    = adigatorCreateAuxInput([2 1]);        % auxiliary input
opts = adigatorOptions('embed_mode','i');    % inline: no .mat, no globals
adigatorGenDerFile_embedded('hessian','gapfun',{w,z},opts);

w = randn(2,1); z = randn(2,1);
[H,G,f] = gapfun_Hes(w,z);   % Hessian, gradient, value
```

6.2 N-Dimensional Auxiliary Inputs

Although *ADiGator* objects are restricted to two dimensions, the `adigatorCreateAuxInput` utility accepts an N-dimensional size for **auxiliary** (parameter) inputs, declaring a parameter such as `B` of size `m×n×K`. The N-D declaration is folded internally to a two-dimensional array, and trailing-subscript slicing such as `B(:,:,k)`, or the multi-counter form `B(:,:,a,k)`, is translated into the corresponding affine column window of the fold, returning an ordinary two-dimensional object. This covers the common "matrix over time steps (and per actuator)" case without an N-D object type. See `examples/jacobians/ndparam/`.

6.3 Struct Inputs

The variable of differentiation and the auxiliary inputs may be carried as fields of a (scalar) struct, to any nesting depth through scalar-struct fields and cell elements (struct arrays are excluded). The convenience and embedded generators locate the variable of differentiation within the struct, seed its derivative along that access path, and forward the remaining fields unchanged; the generated wrapper accepts the same struct shape as the original function. See `examples/jacobians/structinput/`.

6.4 Runtime Loop Bounds

The `LOOPBOUND` option (Table 1) names input(s) of the differentiated function which act as runtime loop bounds. The derivative file is generated at the maximum trip count (the value passed to `adigator`, used for the analysis), and the matching outer loops are printed with the named input as their bound; loop-exit variables take the union over all iterations. The generated file opens with one `assert(n <= max)` per declared bound, before any other statement. That placement is load-bearing rather than cosmetic: it is what tells MATLAB Coder the upper bound of *every* size that depends on the parameter — including your own `zeros(n,1)`-style allocations — so the file compiles with static memory allocation (no heap), as an embedded target requires. The generated file may then be evaluated for any $1 \le \mathtt{n} \le \mathtt{max}$ without regeneration. Results agree with the true `n`-sized program when the post-loop code is *padding-benign* (sums, dot products, scatter/gather over the loop-written entries); operations that read a fixed-size buffer's padded tail (`length`, `end`, `mean`, `max`) see the maximum size. See `examples/jacobians/loopbound/`.

6.5 Alternative Output Forms and Reverse Mode

Several routines target first-order embedded solvers, which want the nonzero vector or matrix-vector products rather than a dense projection:

- **CSC value-vector output (`DER_OUTPUT`).** With `DER_OUTPUT = 'csc'` (Table 1) a generator returns its *top-order* derivative's structurally-possible-nonzero vector in compressed-sparse-column order (values only), exporting the constant pattern once as the per-role `output.*CSC` companion, with no per-call dense allocation or scatter. This covers the **Jacobian** (`output.JacobianCSC`, via `adigatorGenJacFile`) and the **Hessian** (`output.HessianCSC`, via `adigatorGenHesFile`); on a Hessian file it flips only the Hessian output, the gradient companion staying dense. An embedded solver consumes the value vector with the constant `ColPtr`/`RowIdx` directly; the host helpers `adigatorCSCToLocs`/`adigatorCSCToSparse` rebuild coordinate/sparse forms when needed.
- **Matrix-free reverse mode.** The transposed-Jacobian-vector product ($\mathbf{J}^\top\mathbf{v}$) and the scalar-cost adjoint gradient are provided as the black box routines `adigatorGenJtVFile` and `adigatorGenRevGradFile`, documented in Section 5.8. Within the embedded generator, the same adjoint gradient is produced as

an **embeddable** file by passing `DerType = 'gradient-reverse'` to `adigatorGenDerFile_embedded`, which routes the adjoint through the classic/inline pipeline exactly like the forward modes.

6.6 Additional Overloaded Operators

This fork adds the following overloads:

- `norm` — vector p-norms ($p = 2$, 1, ∞, $-\infty$, general p) and the Frobenius norm (`'fro'`) are rewritten to elementary differentiable operations. The induced/spectral matrix norms (which would require a singular-value decomposition) raise a clear error rather than returning an incorrect derivative.
- `isnan`, `isinf`, `isfinite` — derivative-free predicates evaluated on the value at run time, usable in conditionals and masks.

6.7 Which Embed Mode and Derivative Form Should I Pick?

The two embed modes return identical numbers, so the choice is about the generated artifact. In short: use inline `'i'` (the default for `adigatorGenDerFile_embedded`) for embedded code generation — it is self-contained (no `.mat`, no `load`, no globals) and code-generates under MATLAB Coder / Embedded Coder; use classic `'c'` for interactive/host use where the runtime `load` is convenient. For the derivative *form*, prefer `DER_OUTPUT = 'csc'` when a downstream solver assembles (or never forms) the matrix itself, and the matrix-free products (Section 6.5) when only $\mathbf{J}^\top\mathbf{v}$ or an adjoint gradient is needed.

The structural difference between the modes is concrete: classic `'c'` emits three files (a wrapper, a `.mat` of static data, and the derivative file) that read the `.mat` through a runtime `load` and so run only on a host, whereas inline `'i'` emits a single self-contained file that code-generates to embedded C. Because the classic form cannot code-generate, the on-target cost is a property of the inline artifact, and it is small. Table 2 reports the compiled **ROM footprint** (the Embedded-Coder object's `.text`+`.rdata`, read with `size`), the MEX runtime, and an interpreted numerical finite-difference cost of *ADiGator*-generated inline derivatives — forward and reverse gradient, Hessian, and Jacobian — against hand-written analytic references for the same functions, at a representative small size ($n = 8$). At this size the forward- and reverse-mode gradient footprints *converge* and sit modestly above the analytical floor, with zero static RAM: the embeddable forms carry essentially no static data, so automatic differentiation costs little in the embedded artifact. (The compiled ROM is the reliable measurement; the generated-*source*-byte count is a boilerplate-dominated proxy — dominated by comments and `initialize`/`terminate` scaffolding — and is not a footprint.) Numerical finite differences (`FD`) appear as their own compiled row — the same central-difference derivative, written Coder-compatibly and code-generated through Embedded Coder like every other method. Because FD evaluates the cost n times, its compiled ROM is a *multiple* of the hand-derivative's (4.8× for the gradient, 9.0× for the Hessian at $n = 8$); it is the only *inexact* method (central-difference truncation), and its evaluation cost grows $O(n^2)$ against automatic differentiation's $O(n)$, so its disadvantage *grows with problem size*. It is the method one reaches for by default when unwilling to hand-derive, and the table makes the on-target cost of that convenience concrete. The ROM footprint is a deterministic build measurement; the `MEX/ana` runtime ratios are machine-dependent and noisy — the compiled evaluations take microseconds, so the ratio of two tiny timings is dominated by call overhead — and should be read for their *scaling* (see the figure in `bench/SHOWCASE.md`), not as precise figures. The measurement environment (machine, processor, MATLAB / Coder / compiler versions) is stamped in the table source. A broader comparison of the *generated-code complexity* across every axis (embed mode × slim × rolled/unrolled × forward/reverse) is maintained in `bench/SHOWCASE.md`.

The table above is the *compiled*, code-generation-time comparison. A lot of use — prototyping, or a design-time simulation loop — runs derivatives *interpreted*, never compiling; there the ranking shifts (the embed-mode data-load cost and `slim`'s dead-code trimming matter interpreted in ways the C compiler flattens away). Table 3 reports the un-gated interpreted view of the same four methods: the generated-code **complexity** (code lines, deterministic), the interpreted derivative-evaluation time as a ratio to the analytical reference, and the accuracy (`max err`). The accuracy column is where finite differences stand apart: automatic differentiation and the hand derivative are machine-eps exact (0), while FD carries a central-difference truncation error ($\sim 10^{-10}$ for a gradient, $\sim 10^{-8}$ for a Hessian). The complexity columns

function	derivative	method	ROM (B, n=8)	ROM / ana	MEX / ana
vcostfun	gradient	AD (fwd)	208	1.30	1.1
vcostfun	gradient-reverse	AD (rev)	208	1.30	1.1
vcostfun	gradient	analytic	160	1.00	1.0
vcostfun	gradient	FD (num)	768	4.80	1.6
vcostfun	hessian	AD	224	1.00	0.8
vcostfun	hessian	analytic	224	1.00	1.0
vcostfun	hessian	FD (num)	2016	9.00	8.3
vvecfun	jacobian	AD	224	1.27	1.0
vfun	jacobian	AD	448	—	—
vvecfun	jacobian	analytic	176	1.00	1.0
vvecfun	jacobian	FD (num)	432	2.45	1.5

Table 2: Compiled (inline embed) derivatives of the four methods — AD forward, AD reverse, hand-written *analytical*, and numerical finite differences (*FD*) — at $n = 8$, each ERT-compiled to a real footprint: compiled **ROM** (Embedded-Coder `.text`+`.rdata`) and its ratio to the analytic form, plus the `MEX/ana` compiled-runtime ratio. FD is heavier (it evaluates the cost n times) and the only inexact method. The `ROM` columns are deterministic build measurements; the `MEX/ana` ratios are machine-dependent and noisy — read them for scaling (numerical FD grows $O(n^2)$, AD $O(n)$; see the figure), not as precise figures. Rolled `vfun` has no analytic counterpart, hence the dashes. Regenerated by `bench/derivShowcaseC.m`.

are deterministic; the runtime ratios are machine-dependent (measured on the environment stamped in the table source) and should be read as trends.

function	derivative	method	code lines	interp / ana	max err
vcostfun	gradient	AD (fwd)	22	63.8	0
vcostfun	gradient-reverse	AD (rev)	18	10.1	0
vcostfun	gradient	analytic	4	1	0
vcostfun	gradient	FD (num)	3	19.7	3e-10
vcostfun	hessian	analytic	5	1	0
vcostfun	hessian	FD (num)	4	66.6	8e-08
vvecfun	jacobian	analytic	4	1	0
vvecfun	jacobian	FD (num)	3	10.5	3e-11

Table 3: Interpreted (no Coder) view of the four methods at $n = 6$: generated-code **complexity** (non-comment `code lines`), the interpreted derivative-evaluation time as a ratio to the analytical reference (`interp/ana`), and the derivative error vs. that reference (`max err`). AD and analytical are machine-eps exact; *FD* is the only inexact method (central-difference truncation). For AD/analytical, `code lines` is the full implementation; for *FD* it is the per-anchor wrapper only (which reuses a shared `fdDeriv` kernel, not counted), i.e. the marginal per-function authoring cost. Complexity is deterministic; the runtime ratios are machine-dependent — read them as trends, not precise figures. Regenerated by `bench/derivShowcase.m`.

7 User Function File Coding Restrictions

The *ADiGator* package generates derivative code by both reading the user's code and evaluating sections of the user's code on overloaded **cada** objects. This allows it to generate stand-alone derivative files, but due to the complexity of the process, certain coding restrictions are placed on the files which it can differentiate. These are mostly in place for two reasons, 1. that the package can follow the flow of the user's code and track all variables from within it, and 2. that the code has enough information to print valid derivative calculations. In this section we go over the various coding requirements which the user should follow to ensure that their function files are differentiable using *ADiGator*.

7.1 Numeric Outputs (Derivative-File Generators)

Note. The derivative-file generators — `adigatorGenJacFile`, `adigatorGenHesFile`, and `adigatorGenDerFile_embedded` — require the differentiated function to return a **single numeric array** output. A function that returns a `struct` (or cell) is **not supported** by these generators and currently raises an internal error, because a Jacobian/Hessian of a struct-valued output is not a single matrix. Struct/cell *inputs* that carry the variable of differentiation are fully supported (Section 6.3); it is only struct *outputs* through these generators that are unsupported. The lower-level `adigator` command itself *can* differentiate a struct-returning function — its output is then a struct of derivative structs — but the matrix-output wrappers cannot assemble their result from it.

7.2 Known Numeric Values

The overloaded class which the package uses to perform the actual derivative computations is based off of having fixed, known sizes and sparsity patterns. For this reason the user's code must be written such that, given how the inputs are defined and given to the `adigator` command, all variables created within the user program must have a fixed size. As an example, consider a user function `y = myfun(x,N)` which contains the statement `y = zeros(N,1)`. If one were to identify the input `N` as an **Unknown Auxiliary Input**, then this would produce an error as this says that the variable `y` may take on *any* size. If, however, the input `N` was defined as a **Known Auxiliary Input** with value 10, then the derivative code would be generated as if the variable `N` was always 10. Similarly, all referencing/assignment indices must be known values, i.e. if the user code contains a statement `y(J) = x(I);`, then `adigator` must know the values of both `I` and `J`. Furthermore, these values must not change when evaluating the derivative file, or else the derivative calculations will be invalid. The sole exception to this rule is the use of unknown logical referencing and assignment, but this too has certain syntax requirements which are covered in Section 7.6.

7.3 Flow Control

A great deal of work has gone into the ability of the *ADiGator* package to be able to maintain the flow of the user's program within the derivative program. In this section we present the four different types of MATLAB flow control and how the *ADiGator* package deals with them.

7.3.1 Conditional `if` Statements

The user should be able to write any conditional `if` statements within their program, and any *possible* branches of the conditional block will be transcribed to the derivative program. For example, if a user function `y = myfun(x)` is given to `adigator`, the input `x` is defined as a **Derivative Input** with size [10, 1], and the function contains the following:

```
if x(1) > 1
    {calcs1}
elseif length(x) > 10
    {calcs2}
else
    {calcs3}
end
```

then `adigator` would see that `x(1)` may be any value, thus the first branch may or may not happen, that `length(x) = 10`, and thus the second branch would never happen, so the derivative code would be produced along the lines of:

```
if x(1) > 1
    {deriv calcs1}
else
    {deriv calcs3}
end
```

All outputs of conditional fragments should, however, be the same size no matter which branch is taken.

7.3.2 `for` Loop Statements

The user may write any `for` loop statements as long as the loop is set to run for a fixed, known, number of iterations. The default way of handling loops is to keep them rolled in the derivative program (that is, write out the loop), but the user may use the UNROLL option if they wish to unroll the loop. There are tradeoffs which occur when either rolling or unrolling. Namely, if the user chooses to unroll a loop which runs for many iterations, then the generated files can become extremely large as unrolling the loop will make the package print out derivative calculations for each iteration of the loop. Unrolling the loop, however, sometimes can result in a more efficient derivative file, especially if the variables within the loop change size on each iteration of the loop. Here we also note that the use of `break` and `continue` statements is allowed within loops *only* if the loops are to be rolled in the derivative file.

7.3.3 `while` Loop Statements

With the release of *ADiGator* V1.0, `while` loops may be used in the user code under certain restrictions. Namely, the `while` loops should not contain any iteration dependent conditional statements or organizational operations. The primary purpose of allowing `while` loops is for performing Newton-type iterations and thus the following code is fine:

```
count = 0;
xk    = x;
fk    = myfun(xk);
Jk    = myjac(xk);
 while norm(fk) > 1e-6 && count < 100
     count = count+1;
     xk    = xk - Jk\fk;
     fk    = myfun(xk);
     Jk    = myjac(xk);
 end
```

If a `while` loop contains any organizational operations which are dependent upon `while` loop iterations, then the code cannot be differentiated and the `while` loop should be replaced with a `for/if/break` sequence. For example, the following `while` loop

```
count = 1;
xi = x(1);
 while xi < 0
    count = count+1;
    xi = x(count);
end
```

cannot be differentiated due to the iteration dependent reference operation (`x(count)`). It should be replaced with the `for` loop:

```
 for count = 1:length(x)
    xi = x(count);
    if xi >= 0
        break
    end
end
```

The algorithm will attempt to find a static loop iteration where input sparsity patterns are equal to output sparsity patterns. There exists a maximum number of iterations for which it will attempt to find said fixed iteration. The user may change this maximum iteration limit using the `adigatorOptions` command (option `MAXWHILEITER`).

7.3.4 `switch` Statements

`Switch` statements are not currently allowed, please use `if` statements instead.

7.3.5 Short Circuit AND/OR

Here we note that using the short-circuit `and` (`&&`) and short-circuit `or` (`||`) are allowed, but they will be replaced with the non-short-circuit `and` (`&`) and non-short-circuit `or` (`|`) commands in the intermediate program. Thus, if the user has a line of code such as

```
if {statement1} && {statement2}
```

this will be replaced by

```
if {statement1} & {statement2}
```

thus, if evaluating `statement2` will produce an error given that `statement1` is false, then the program will not be able to be differentiated. Similarly, if the short-circuit `or` command is used such that evaluating the second statement produces an error given that the first statement is true, then the program will not be able to be differentiated.

7.4 Functions and Sub-Functions

The user's main function file given to the `adigator` command is allowed to call as many other functions/sub-functions as desired. Each called function will be opened, read, and treated as if it were a `for` loop, thus if the UNROLL option is set 1 and a sub-function is called multiple times, then multiple sub-functions will be printed to the derivative file. The restrictions on called functions and sub-functions are as follows. First, the function must be in the MATLAB path. Second, called functions and sub-functions should not share the same name, this will produce an error. Third, functions should not call themselves from within their own methods, this can create an infinite loop within the *ADiGator* algorithm and cause an error. Fourth, currently only *one* function call is allowed per line, if the user has multiple function calls on the same line then an error will be produced. Finally, each function called must have at least one input and one output. The use of the `feval` command is frowned upon here, but will work in some cases. Namely, if the function being called by the `feval` command contains no flow control and is not called from within flow control statements.

7.4.1 MEX Functions NOT Allowed

Unfortunately *ADiGator* cannot differentiate functions which contain calls to MEX files as the algorithm is based off of reading code written in the MATLAB language.

7.4.2 Functions Outputting Different Number of Outputs (or Different Size Cell/Structure Arrays) Based on a User Flag

Often times user's will have function files which are to perform differently depending upon some set user flag, where, perhaps the flag tells the function which model is being used. If, for instance, one model returns an output structure array with one element, and the other model returns an structure array with two elements, then the *ADiGator* algorithm is likely to error when attempting to union the two outputs together. An example of this would be:

```
if userflag
    y(1).v = f(x);
    y(2).v = g(x);
else
    y.v = h(x);
end
```

where the structure `y` is an output. This is likely to produce an error as the two different branches essentially are made to solve two completely different systems. So, at this point in time, it is best to create two different function files, one for each case, and to then generate two different derivative files via *ADiGator*.

7.5 Non-Input Auxiliary Data: Global/Persistent Variables and the Load Command

A user may wish to obtain auxiliary problem data using either global variables or the MATLAB `load` command. Both of these are acceptable, with some stipulations. Namely, if either is used, they should *only* be used to give auxiliary data to the user's function file, they should *not* be used to pass variables between called functions in the program being differentiated. The *ADiGator* package treats all global variables and loaded in variables as if they are **Known Numeric Inputs**, and thus all global parameters should be set to the values that they will be at the time of derivative evaluation. The last restriction is that, when using the `load` command, it must be of the syntax `var = load(.)` and not simply `load(.)`, as for the second case the algorithm would be unable to track what data was brought into the program. Additionally, for the time being persistent variables are not allowed, these are best replaced by global variables with the load sequence being performed outside of the function file.

Embedded modes. The allowances above (`global`/`load` treated as known auxiliary data) apply to the classic `'c'` mode. When generating an **embeddable** file — `adigatorGenDerFile_embedded`, or any `EMBED_MODE` other than `'c'` — a `global` variable, the `load` command, or a cell array in the differentiated function makes the generated file **not self-contained**: embedded modes are no more restrictive than classic, so these are emitted verbatim (exactly as in classic mode) and generation proceeds, but a `adigator:embed:unsupportedConstruct` **warning** names the offending line and notes the file may not code-generate under MATLAB Coder until the construct is removed. To make the file embeddable, pass such data as struct or numeric auxiliary inputs instead, pre-loading any data outside the function. Constructs that classic itself rejects (a bare `load(.)` with no output, unsupported cell patterns) still raise the usual error.

7.6 Unknown Logical Referencing/Assignment

As stated in Section 7.2, the *ADiGator* algorithm is based off of having fixed, known, variable sizes. This presents an issue when dealing with unknown logical referencing and assignments, as the result of a logical reference may take on a number of different sizes depending upon the reference index. To account for this, we require that, if an unknown logical reference/assignment index is used, the same indexing variable must be used for both a reference and assignment. That is, the logical reference/assignment must be of the form:

```
ind = x > 1;
y(ind) = x(ind);
```

Furthermore, if any binary mathematical array operations (+,-,etc.) are to be performed on a variable resulting from a logical reference, then both inputs to the binary operation must be result from a logical reference of the same index. For example,

```
ind = x > 1 & x <2;
zi = x(ind).*y(ind);
z(ind) = zi;
```

is valid,

```
z(x > 1 & x <2) = x(x > 1 & x <2).*y(x > 1 & x <2)
```

is not, even though they perform the same operations.

8 Higher Order Derivatives

A nice outcome from the fact that the *ADiGator* algorithm creates stand-alone derivative code (aside from evaluation speed) is that the method is fully repeatable. Thus, the user may create n^{th} order derivative files. In order to do this, the process is repeated just as shown in Section 3, except now you are differentiating a previously created derivative file. To illustrate, suppose we wished to take a second derivative of a function `y = myfun(x)`, with respect to the input `x`, where $\mathbf{x} \in \mathbb{R}^{10}$, and we want to call our VOD `'x'`. We would first create the first derivative file as follows:

```
x = adigatorCreateDerivInput([10 1],'x');
adigator('myfun',{x},'myderiv1');
```

If we then wished to take a second derivative with respect to $\mathbf{x}$, we would need to define a new input as the input structure to `myderiv1` is different from `myfun`. This would be done as:

```
x = struct('f',x,'dx',ones(10,1));
```

which says that the input `x.f` has the same derivatives as we defined previously, and that the input `x.dx` is a vector of ones which has no derivatives with respect to $\mathbf{x}$ (the second derivative of $\mathbf{x}$ with respect to itself is zero). We can then generate the second derivative file using the command

```
adigator('myderiv1',{x},'myderiv2');
```

In order to evaluate this function, `myderiv2`, we note that the input is exactly the same as the input to `myderiv1` since we have defined no new derivatives. So, we could evaluate it at a set of random points using the commands:

```
x.f = rand(10,1);
y = myderiv2(x);
```

Now, assuming the output has second order derivatives, the output `y` would have the following fields: `.f, .dx, .dx_size, .dx_location, .dxdx, .dxdx_size, .dxdx_location` corresponding to the function value, first derivative value, first derivative array size, first derivative mapping indices, second derivative value, second derivative array size, and second derivative mapping indices, respectively. Assuming the output `y.f` is a scalar, we could then build a sparse Gradient and Hessian using the commands:

```
Grad = sparse(ones(length(y.dx),1),y.dx_location,y.dx,1,y.dx_size);
Hes = sparse(y.dxdx_location(:,1),y.dxdx_location(:,2),y.dxdx,...
             y.dxdx_size(1),y.dxdx_size(2));
```

This process may be repeated as many times as desired.

Here we note that the *ADiGator* algorithm is cognizant of when it is differentiating a function which it created, thus a bunch of embedded structures are not required for the inputs/outputs of the higher order derivative files. Furthermore, it knows what derivatives were taken on previous transformations and identifies these using the string name given to the variable of differentiation. This allows the user to take derivatives with respect to different variables if it is desired, but if the same VOD name is used as a previous transformation, then the given **Derivative Inputs** should reflect those given in the previous transformation.

Here we also note that the user can write functions which call previously created derivative files and then differentiate the new file and that the algorithm will still recognize the previously created derivative files as their own and differentiate accordingly.

9 Differentiating Vectorized Code

The use of the vectorized differentiation is for problems of the form $\mathbf{f}(\mathbf{x}(s))$, where s is a scalar, independent variable. It is easiest to think of s as being a representation of time such that $\mathbf{f}$ at $s = t$ is only a function of $\mathbf{x}$ at $s = t$. If this is the case, and a user's code is written such that it computes the values of $\mathbf{f}$ at a set of time points, given the inputs of $\mathbf{x}$ at a set of time points, then the code may be differentiated in a vectorized manner. More specifically, if the code is written to compute $\mathbf{F}(\mathbf{X}(\mathbf{s})) : \mathbb{R}^{N\times n} \to \mathbb{R}^{N\times m}$, where

$$\mathbf{X}(\mathbf{s}) = \begin{bmatrix} x_1(s_1) & \cdots & x_n(s_1) \\ x_1(s_2) & \cdots & x_n(s_2) \\ \vdots & \ddots & \vdots \\ x_1(s_N) & \cdots & x_n(s_N) \end{bmatrix}, \quad \mathbf{F}(\mathbf{X}(\mathbf{s})) = \begin{bmatrix} f_1(\mathbf{x}(s_1)) & \cdots & f_m(\mathbf{x}(s_1)) \\ f_1(\mathbf{x}(s_2)) & \cdots & f_m(\mathbf{x}((s_2)) \\ \vdots & \ddots & \vdots \\ f_1(\mathbf{x}(s_N)) & \cdots & f_m(\mathbf{x}((s_N)) \end{bmatrix}, \tag{2}$$

where it is important that the code be written such that N may be *any* positive integer. When this is the case, we compute the Jacobian $\mathbf{Jf}(\mathbf{x}(s))$, but do so such that the non-zero elements of $\mathbf{Jf}(\mathbf{x}(s))$ are computed for N values of s, where, again, N may take on any integer value.

9.1 Illustrative Example of Vectorized Differentiation

In order to differentiate in the vectorized mode the user must simply identify their vectorized inputs. To do this, the vectorized dimension is just given the value `Inf`. So, if the user has a function, `Y = myfun(X)`, of the above form with $n = 3$, $m = 4$, then to generate the **Derivative Input**, the following command would be used:

```
X = adigatorCreateDerivInput([Inf 3],'X');
```

To then create a vectorized derivative file, `myderiv`, we would call

```
adigator('myfun',{X},'myderiv');
```

In order to now evaluate the file `myderiv` we must define the function field of `X.f` and the derivative field `X.dX`. Supposing we wish to evaluate at a set of random function inputs at $N = 10$ values of s, we would define the function field as

```
x.f = rand(10,3);
```

Now, here we note that the derivative field needs to be of the dimension $N \times nz$, where nz corresponds to the number of non-zeros of the input Jacobian (in this case $\mathbf{Jx}((\mathbf{x}(s)))$, and N corresponds to the vectorized dimension. In this case, $nz = 3$, and

$$\frac{\partial x_i(s)}{\partial x_j(s)} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \quad \forall s \tag{3}$$

thus the derivative field should be defined as

```
x.dX = ones(10,3);
```

We may then evaluate the derivative function using the command

```
y = myderiv(x);
```

Now, the output fields of $\mathbf{y}$ will have the same form as in the non-vectorized case, with a few exceptions. First, the values assigned to `y.dX_size` and `y.dX_location` are the size and non-zero locations of the Jacobian $\mathbf{Jy}(\mathbf{x}(s))$ evaluated at a single time point. Next, the derivative values stored in `y.dX` are stored in an array of size $N \times nz$, where nz is the number of possible non-zero derivatives in the Jacobian $\mathbf{Jy}(\mathbf{x}(s))$ evaluated at a single point. So, to build the Jacobian $\mathbf{Jy}(\mathbf{x}(s))$ evaluated at $s = s_1$, we could do so as

```
Jac1 = sparse(y.dX_location(:,1),y.dX_location(:,2),y.dX(1,:),y.dX_size(1),y.dX_size(2));
```

9.2 Projecting Vectorized Derivatives into an Unrolled Jacobian

Now, if we let $x_i(\mathbf{s})$ $(i = 1, \ldots, n)$ and $f_j(\mathbf{X}(\mathbf{s}))$ $(j = 1, \ldots, m)$ be column vectors of length N, where the k^{th} elements are $x_i(s_k)$ and $f_j(\mathbf{x}(s_k))$ $(k = 1, \ldots, N)$, respectively, we can then define

$$\mathbf{x}^\dagger(\mathbf{s}) = \begin{bmatrix} x_1(\mathbf{s}) \\ x_2(\mathbf{s}) \\ \vdots \\ x_n(\mathbf{s}) \end{bmatrix}, \quad \mathbf{f}^\dagger(\mathbf{x}^\dagger(\mathbf{s})) = \begin{bmatrix} f_1(\mathbf{X}(\mathbf{s})) \\ f_2(\mathbf{X}(\mathbf{s})) \\ \vdots \\ f_m(\mathbf{X}((\mathbf{s})) \end{bmatrix}. \tag{4}$$

Now, supposing the user wished to build the unrolled Jacobian $\mathbf{Jf}^\dagger(\mathbf{x}^\dagger(\mathbf{s}))$, we have supplied the function `adigatorProjectVectLocs`, which takes the output locations of the non-zeros of the Jacobian $\mathbf{Jf}(\mathbf{x}(s))$, together with the length of the vectorized dimension, N, to build the non-zero locations of the unrolled Jacobian $\mathbf{Jf}^\dagger(\mathbf{x}^\dagger(\mathbf{s}))$. For the above example, we could build the unrolled Jacobian follows:

```
[I,J] = adigatorProjectVectLocs(10,y.dX_location(:,1),y.dX_location(:,2));
Jacdag = sparse(I,J,y.dX,y.dX_size(1)*10,y.dX_size(2)*10);
```

Here we note that one should be careful when using this command as it will only be correct if the input's first dimension is vectorized and the output's first dimension is vectorized. That is, $\mathbf{x}^\dagger(\mathbf{s})$ would correspond to `x.f(:)` and $\mathbf{f}^\dagger(\mathbf{x}^\dagger(\mathbf{s}))$ would correspond to `y.f(:)` in our example, due to how we have arranged the data.

9.3 Vectorized Coding Restrictions

The first vectorized coding restriction is that, if the vectorized dimension of the input is N, then all vectorized variables must be of size $N \times n$, where n must be a known value (and not equal to N). Furthermore, one may not sum over any vectorized dimension, as this results in derivatives of the output being dependent upon all points in time. Similarly, referencing off of the vectorized dimension, but not taking the entire row/column is not allowed, e.g. if `X` is of size $N \times 3$, you cannot perform `xi = X(1,:)`, but you can perform `xi = X(:,1)`. Finally, you are not allowed to loop on a vectorized dimension, e.g. if `Y` is of size $1 \times N$, you may not do `for yi = 1:Y`. It is noted here that unknown logical referencing/assignments are coded up for vectorized dimensions and that these may be used (in accordance with the guidelines given in Section 7.6) instead of a `for` loop if you need to search through the vectorized dimension.

10 Generating Files for GPOPS II

Note. Like the other `adigatorGenFiles4*` wrappers (Section 5.3), `adigatorGenFiles4gpops2` is inherited from upstream *ADiGator* and is **deprecated** — unmaintained, and **not at feature parity** with this fork's embedded pipeline (host-only; no `EMBED_MODE`/code generation/`PATH`/`csc`/reverse mode). It is retained for upstream continuity only and may lose functionality in a future release.
GPOPS II is a commercial MATLAB optimal control software which utilizes direct collocation methods to solve optimal control problems. The creators of *ADiGator* have collaborated with those of *GPOPS II* in order to allow for *ADiGator* to supply first- and second-order derivatives to *GPOPS II*. In order to generate the derivative files using *ADiGator* please see the file `adigatorGenFiles4gpops2`. This function utilizes the vectorized and non-vectorized modes in order to generate the derivative files required by *GPOPS II* given the `setup` structure which is used by the `gpops2` function. Like all *ADiGator* generated files, these generated files will not need to be changed unless the user changes their continuous or endpoint functions.

11 Debugging

During the transformation from user code to derivative code, the *ADiGator* algorithm never actually evaluates the user's code, but rather copies various parts of it, writes them to another file, and then evaluates that file. This can sometimes make debugging difficult, particularly because the MATLAB error messages do not point to the user's file with the errors. Rather, they will usually point to a user's line of code which was copied to another file. We are working on a way to point to the user's file, but for now the user will need to deal with the inconvenience. In this section we will go over some of the common errors that may occur when using the *ADiGator* software.

11.1 Error in User Code or Inputs to ADiGator

Prior to performing any transformation, the algorithm first attempts to evaluate the user's function numerically on what it believes to be the inputs. If you receive the error

```
Error in initial test evaluation of user file on given input info
```

then one of two things has occurred: either there is an error in the user's file or the inputs to the user's file have not been properly given to `adigator`. To debug this, you should first ensure that your function evaluates on numeric inputs without errors. If you are still receiving the error, then refer to Section 3.1.2 for the proper input scheme.

11.2 Non-Overloaded Functions

The algorithm evaluates the user's statements on an overloaded **cada** class, so, if the user's code contains calls to functions which are not overloaded an error will be produced as

```
Undefined function 'somefunction' for input arguments of type 'cada'.
```

In this case, the only way to use that function within the user's code is to overload the function. If the calculation in question may be performed prior to the user's code (i.e. you do not need to take derivatives of the function), then this is a simple solution. If however, the function in question is operating on objects which have derivatives, then the user may either contact the author in order to see if the function can be overloaded, or rewrite their code to use a different function. A list of all the currently overloaded operations may be seen by typing "`help cada`" and then clicking on the link to the reference page.

11.3 Errors Regarding "Strictly Symbolic" Inputs

As stated in Section 7.2, in order to create an array, the size of the array must be a known numeric value. Similarly, in general, when performing a reference/assignment, the reference/assignment index must be a known numeric value. When these types of operations are performed and the algorithm does not know the value of the size and/or index, then an error will be produced. For example, if one were to perform the operation `y = ones(N,1)`, and the variable `N` does not have a known numeric value, then the error

```
Cannot pre-allocate a stricly symbolic size
```

would be produced. Similarly, if one were to perform the operation `y = x(I)` and the variable `I` does not have a known numeric value, then the error

```
Cannot perform strictly symbolic referencing/assignment
```

would be produced. In order to fix this, ensure that, if these sizes/indices are inputs, that they are given such that they are **Known Auxiliary Inputs** as shown in Section 3.1.2. If they are values calculated from within the file, make sure that they are created by operating on known numeric objects. Also, note that the sizing operations `size,numel,length` will always produce a known numeric value.

When the index is genuinely *data-dependent* — computed from runtime values rather than resolvable to a fixed pattern (for example a subscript derived from a `while`-loop counter or from the numeric value of an input) — the operation cannot be handled by static forward AD, and the tool raises an *actionable* error that names the offending construct and points to a fixed-subscript rewrite — a constant logical mask reused for both the reference and the assignment, or the equivalent logical-weighted sum, in the spirit of Section 7.6 — as the supported alternative, rather than silently producing a wrong derivative.

11.4 Errors in the Derivative and/or Derivative Code

If you receive an error in the evaluation of the derivative code (on proper numerical inputs), then you have likely found a bug in the algorithm. Please report this. Similarly, if a derivative is computed incorrectly, then you have more than likely found a bug in the algorithm. Please report this as well. If an element of the derivative is being computed as `NaN`, then the derivative is likely undefined at the evaluation point. A common occurrence of this is evaluating the derivative of the `sqrt` function at 0 when the derivative of the `sqrt` argument is also zero. That is, if `y = sqrt(x)`, the derivative rule is produced along the lines of `dy = (1/2)/sqrt(x)*dx`; if this is then evaluated when `x=0` and `dx=0`, then `1/sqrt(x)` goes to infinity, and zero times infinity is undefined. Thus, the derivative code isn't wrong, the derivative is just undefined at that point.

12 List of Included Examples

In this section we present an explanation of the examples included with the *ADiGator* package. To run all the examples in this section, change your working directory to the 'examples' directory and type 'runAllExamples'.

12.1 Basic First Derivatives

In this section we present two examples which take the first derivative of vector functions of vectors and compare against MATLAB's `numjac` finite differencing tool.

12.1.1 Arrowhead

This is an example taken from Section 7.4 of [3]. Here we take the derivative of a function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^n$, where

$$f_1(\mathbf{x}) = 2x_1^2 + \sum_{i=1}^{n} x_i^2, \quad f_j(\mathbf{x}) = x_1^2 + x_j^2, \quad (j = 2, \ldots, n) \tag{5}$$

This is coded up such that the Jacobian, $\mathbf{Jf}(\mathbf{x})$ is built using *ADiGator*, finite differencing, and compressed finite differencing. Here the user is free to change the the parameter `N` within the `main.m` (located in `examples/jacobians/arrowhead/`)code in order to see that the *ADiGator* algorithm becomes more useful as the problem size increases. Furthermore, you can see that there is a certain amount of overhead inherent with generating the derivative code, but, if you wish to evaluate the code many times, this overhead becomes less of an issue.

12.1.2 Polynomial Data Fitting

This example was taken from [6] which is to determine the coefficients of the m-degree polynomial $p(x) = p_1 + p_2 x + p_3 x^2 + \cdots + p_m x^{m-1}$ that best fits the points (x_i, d_i), $(i = 1, \ldots, n)$, $(n > m)$, in the least squares sense. The code may be found in `examples/jacobians/polydatafit`. This polynomial data fitting problem leads to an overdetermined linear system $\mathbf{Vp} = \mathbf{d}$, where $\mathbf{V}$ is the Vandermonde matrix,

$$\mathbf{V} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{m-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{m-1} \\ \vdots & \vdots & & \vdots & \\ 1 & x_n & x_n^2 & \cdots & x_n^{m-1} \end{bmatrix}. \tag{6}$$

As with the previous example, the user should note that the overhead associated with generating the derivative file becomes less as the problem size increases, as well as the number of required derivative evaluations. Unlike the previous example, it is noted that the resulting Jacobian, $\mathbf{Jp}(\mathbf{x})$, is full (i.e. has no known non-zero elements), thus, after the derivative file is evaluated, the Jacobian is simply built using the reshape command:

```
dpdx = reshape(p.dx,p.dx_size);
```

12.2 Stiff Ordinary Differential Equations

In this section we supply derivatives to the MATLAB ODE integrator, `ode15s`, to integrate two different ODEs.

12.2.1 Brusselator

In this example, we integrate the well known Brusselator system of [7]. The dynamics of the system are given as

$$\begin{aligned} \dot{u}_i &= 1 + u_i^2 v_i - 4u_i + \alpha(N+1)^2(u_{i-1} - 2u_i + u_{i+1}) \\ \dot{v}_i &= 3u_i - u_i^2 v_i + \alpha(N+1)^2(v_{i-1} - 2v_i + v_{i+1}) \end{aligned} \quad (i = 1, \ldots, N) \tag{7}$$

with initial conditions

$$\begin{aligned} u_i(0) &= 1 + \sin\left(2\pi \tfrac{i}{N+1}\right) \\ v_i(0) &= 3 \end{aligned} \quad (i = 1, \ldots, N) \tag{8}$$

The code contained in `examples/stiffodes/brusselator/main.m` will generate the derivative file for the system using *ADiGator*. It then integrates the system with `ode15s` three different times. The first time it does not supply derivatives, the second time it supplies the sparsity pattern (as calculated from the outputs of the `adigator` transformation command), and finally it supplies derivatives using the *ADiGator* generated file. Here we note that the *ADiGator* generated derivative file may not be given directly to `ode15s` as the input/output scheme of the generated files is different from the required input/output scheme for use with `ode15s`. Thus, a wrapper file is written such that the wrapper file has the input/output scheme required of `ode15s`. The wrapper file then uses its inputs to call the generated derivative file, and then build the

Jacobian output using the outputs of the derivative file. The user is free to change the dimension N and the time span over which to integrate. They should notice that supplying no derivative information is extremely slow, while supplying just the sparsity pattern or the sparsity pattern plus derivatives yield similar solve times.

12.2.2 DCAL Control of Two-Link Robot Manipulator

This example is a simulation of the experiment presented in [8]. The paper derives a control for the robotic model:

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{V}_m(\mathbf{q}, \dot{\mathbf{q}})\mathbf{q} + G(\mathbf{q}) + \mathbf{F}_d(\dot{\mathbf{q}}), \tag{9}$$

where $\mathbf{q}(t)$ is a position vector. The control, $\boldsymbol{\tau}$, is computed as a function of $\mathbf{q}$, $\mathbf{Y}_d$ and $\dot{\mathbf{Y}}_d$, where

$$\mathbf{Y}_d = \mathbf{Y}_d(\mathbf{q}_d, \dot{\mathbf{q}}_d, \ddot{\mathbf{q}}_d) \tag{10}$$

and the vector $\mathbf{q}_d(t)$ is the desired trajectory. For this simulation, we have a two-link robot, thus $\mathbf{q}, \mathbf{q}_d, \boldsymbol{\tau} \in \mathbb{R}^2$, with the desired trajectory given as

$$q_{d1}(t) = q_{d2}(t) = 0.7\sin(2t)\left(1 - e^{-.3t^3}\right). \tag{11}$$

Prior to integrating the differential equation is is required that the time derivatives $\dot{\mathbf{q}}_d$, $\ddot{\mathbf{q}}_d$, and $\dot{\mathbf{Y}}_d$ are derived. Rather than doing this by hand, this is done using *ADiGator* as follows. First, we differentiate the desired trajectory function `qd = getqd(t)` with respect to `t`. We then differentiate the resulting derivative file to get a file for $\ddot{\mathbf{q}}_d$, and then once more differentiate the resulting derivative file to get a file which calculates $\dddot{\mathbf{q}}_d$. We then define the vector $\mathbf{Q} = [\mathbf{q}_d^T, \dot{\mathbf{q}}_d{}^T, \ddot{\mathbf{q}}_d{}^T]^T$, where $\mathbf{Y}_d = \mathbf{Y}_d(\mathbf{Q})$. The calculations to get $\mathbf{Y}_d$ are given in the file `Yd = getYd(Q)`. We then take the derivative of the file `getYd` with respect to time in order to get a file which computes $\dot{\mathbf{Y}}_d(\mathbf{Q}, \dot{\mathbf{Q}})$. The control laws written in the file `getDCALcontrol` are then written such that they call these generated derivative files.

Having generated these time derivative files, we then solve the ODE using `ode15s`. Here we note that we are just integrating the robot positions and velocities, but also an internal filter variable and a variable used in the parameter update law. Thus our states which we are integrating are defined as $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T, \mathbf{p}^T, \mathbf{z}^T]^T$, where $\mathbf{q} \in \mathbb{R}^2$ is the robot link position, $\dot{\mathbf{q}} \in \mathbb{R}^2$ is the robot link velocity, $\mathbf{p} \in \mathbb{R}^2$ is the internal filter variable, and $\mathbf{z} \in \mathbb{R}^5$ is the variable used in the parameter update law. The dynamic equations associated with all variables is calculated from within the `xdot = TwoLinkSys(t,x)` file (which calls `getDCAlcontrol`). We then take the derivative of the file `TwoLinkSys` with respect to `x` and supply this to the ODE solver `ode15s` in order to integrate the system.

As with the previous example, a wrapper is used since the input/output scheme required by `ode15s` is different from that of *ADiGator* generated files. Within the `main.m` file (which runs the simulation), the user is free to change the `supplyderiv` flag (to supply derivatives or not), the `displayplots` flag (to display plots or not), the `probinfo.noiseflag` (to add noise or not), or the value of final time, `tf`. Increasing the final time, not supplying derivatives, or adding noise will all increase the simulation run time.

12.2.3 Burgers' Equation

This example integrates a moving-mesh discretisation of Burgers' equation (a benchmark from the MATLAB `ode15s` documentation) using a state-dependent mass matrix. The integrated state $\mathbf{y} = [\mathbf{a}^T, \mathbf{x}^T]^T$ stacks the nodal values $\mathbf{a}$ and the moving mesh coordinates $\mathbf{x}$. The code in `examples/stiffodes/burgers/` differentiates the loop-free form of the right-hand side, `burgersfun_noloop`, with `adigatorGenJacFile` — the loops are removed because the flattened form transforms more efficiently — and supplies the resulting Jacobian and its sparsity pattern to `ode15s`, alongside the analytic mass matrix and its sparsity pattern. As with the other stiff-ODE examples, a wrapper adapts the generated file to the input/output scheme `ode15s` expects. The user may change the problem dimension `n` and the time span `tspan`.

12.3 Constrained Optimization

In this section we supply derivatives to the MATLAB constrained optimization solver `fmincon` to solve three different problems. The first example is given as a simple demonstration on how to supply Objective Gradients, Constraint Jacobians, and Lagrangian Hessians using *ADiGator*. In the second example we divide our problem into a vectorized portion and a non-vectorized portion. Finally, in the last example, we take full advantage of the vectorized nature of the problem.

12.3.1 Simple Example

The code for this example may be found in `examples/optimization/fminconEx/`. *Note: this example demonstrates a deprecated wrapper (Section 5.3); it is retained for upstream continuity and is not embeddable.* For this example, we use the problem taken from the MATLAB optimization toolbox help documentation. The problem is as follows:

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^2} f(\mathbf{x}) &= e^{x_1}(4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1) \\ &\text{such that} \\ \mathbf{c}(\mathbf{x}) = \begin{array}{l} x_1x_2 - x_1 - x_2 + 1.5 \\ -x_1x_2 - 10 \end{array} &\leq \mathbf{0} \end{aligned} \quad . \tag{12}$$

We use this somewhat simple problem to demonstrate how to create an *objective Gradient*, *constraint Jacobian*, and *Lagrangian Hessian*. Furthermore, we show how these may be used with MATLAB's non-linear programming solver, `fmincon`. If the user does not have the optimization package, then `fmincon` will not be called, but they may still see how to generate the mentioned derivatives. We do this as follows. First, we solve the problem without supplying derivatives using the active-set method of `fmincon`. Next, we generate objective Gradient and constraint Jacobian files and supply these to `fmincon` to solve the problem again with the active-set method. We then move onto generating Lagrangian Hessians and supplying them to `fmincon`. This is done in two ways. The first is to write a file which computes the Lagrangian, and differentiate it twice using *ADiGator*. In the second way, we take advantage of the fact that we previously created derivative files to compute the objective Gradient and constraint Jacobian, thus we write a file which computes the Lagrangian Gradient by calling these files. The Lagrangian Gradient file is then differentiated, achieving the same thing as if we were to simply differentiate the Lagrangian twice. Here we stress that various wrapper files must be made in order to use the *ADiGator* generated files with `fmincon` as the input/output scheme required by `fmincon` is different from the input/output scheme of *ADiGator* generated files. These files are included, but are not automatically generated.

12.3.2 Brachistochrone

In this example, we solve a discretized form of the continuous time optimal control problem:

$$\min_{\mathbf{x}(t),u(t),t_f} t_f \tag{13}$$

subject to the dynamic constraints

$$\mathbf{f}(\mathbf{x}(t),\mathbf{u}(t)) = \begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \dot{x}_3(t) \end{bmatrix} = \begin{bmatrix} x_3(t)\sin u(t) \\ -x_3(t)\cos u(t) \\ g\cos u(t) \end{bmatrix} \tag{14}$$

and boundary conditions

$$\begin{array}{llll} x_1(0) = & 0, & x_1(t_f) = & 2 \\ x_2(0) = & 0, & x_2(t_f) = & -2 \\ x_3(0) = & 0. & & \end{array} \tag{15}$$

To solve this problem we use a Hermite-Simpson Separated collocation method. We do this using K equally spaced intervals with a discrete point at the beginning and middle of each interval, as well as at a final point.

This results in $N = 2K + 1$ discrete time points, t_j $(j = 1, \ldots, N)$. Here we introduce our discretized state, $\mathbf{X} \in \mathbb{R}^{N\times 3}$ where

$$X_{i,j} = x_i(t_j) \quad (i = 1, 2, 3) \quad (j = 1, \ldots, N) \tag{16}$$

and our discretized control, $\mathbf{U} \in \mathbb{R}^N$, where

$$U_j = u(t_j) \quad (j = 1, \ldots, N). \tag{17}$$

We also introduce the matrix function $\mathbf{F}(\mathbf{X}, \mathbf{U}) \in \mathbb{R}^{N\times 3}$, where,

$$F_{i,j} = f_i(\mathbf{X}_j^T, U_j), \quad (i = 1, 2, 3) \quad (j = 1, \ldots, N), \tag{18}$$

where $\mathbf{X}_j$ is the j^{th} row of the matrix $\mathbf{X}$. We now give the discretized problem as:

$$\min_{\mathbf{X},\mathbf{U},t_N} t_N \tag{19}$$

subject to the equality constraints

$$\begin{array}{c} X_{i,2k+1} - \frac{1}{2}(X_{i,2k+2} + X_{i,2k}) - \frac{t_N}{8K}(F_{i,2k} - F_{i,2k+2}) = 0 \\ X_{i,2k+2} - X_{i,2k} - \frac{t_N}{6K}(F_{i,2k+2} + 4F_{i,2k+2} + F_{i,2k}) = 0 \\ (i = 1, 2, 3) \quad (k = 1, \ldots, K) \end{array} \tag{20}$$

and simple bounds

$$\begin{array}{c} \mathbf{X}_{min} \leq \mathbf{X} \leq \mathbf{X}_{max} \\ \mathbf{U}_{min} \leq \mathbf{U} \leq \mathbf{U}_{max} \\ t_{Nmin} \leq t_N \leq t_{Nmax}, \end{array} \tag{21}$$

where we are using the notation that $\mathbf{X}_i$ is the i^{th} row of the matrix $\mathbf{X}$, and that the boundary conditions of Equation 15 are included in the simple bounds of Equation 21. There are five different driver files which will solve this problem in the `examples/optimization/vectorized/brachistochrone` directory. The file `main_noderivs` solves it without supplying derivative information. The files `main_basic_1stderivs` and `main_basic_2ndderivs` solve the problem by supplying first and second derivative information, respectively, without taking advantage of the vectorized nature of $\mathbf{F}(\mathbf{X}, \mathbf{U})$. And the files `main_vect_1stderivs` and `main_vect_2ndderivs` solve the problem by supplying first and second derivatives, respectively, while taking advantage of the vectorized nature of $\mathbf{F}(\mathbf{X}, \mathbf{U})$. Here we note that we do not use *ADiGator* to compute the Objective Gradient as this is a simple calculation that does not require automatic differentiation. Furthermore, all of these files solve the problem four different times using values of $K = 5$, $K = 10$, $K = 20$, and $K = 40$, where, for the last three iterations the solution of the previous iteration is used as an initial guess to the current iteration.

Supplying Derivatives without Vectorization

In order to supply a Constraint Jacobian in the most straightforward way, we write the function `[C,Ceq] = basic_cons(z,probinfo)`, where the input vector is $\mathbf{z} = [\mathbf{X}^{\dagger T}, \mathbf{U}^T, t_N]^T$, where $\mathbf{X}^\dagger$ is the unrolled form of $\mathbf{X}$ (as described in the Appendix). The file `basic_cons` calls the file `F = dynamics(X,U)` and uses the output to build the constraints of Equation 20. So, in order to get the Constraint Jacobian, we simply apply *ADiGator* to the `basic_cons` file using `z` as our variable of differentiation. To then supply the Constraint Jacobian to `fmincon` using `fmincon`'s desired input/output scheme, we write the wrapper `basic_conswrap` which calls our generated derivative file and builds the Constraint Jacobian.

In order to supply the Lagrangian Hessian, we first write a file, `GL = basic_laggrad(z,lambda,probinfo)`, which uses our constraint derivative file to build the Lagrangian Gradient:

$$\bigtriangledown_{\mathbf{z}} L = \bigtriangledown_{\mathbf{z}} t_f + \boldsymbol{\lambda}^T \bigtriangledown_{\mathbf{z}} \mathbf{c}(\mathbf{z}), \tag{22}$$

where $\bigtriangledown_{\mathbf{z}} L$, $\bigtriangledown_{\mathbf{z}} t_f$, $\boldsymbol{\lambda}$, and $\bigtriangledown_{\mathbf{z}} \mathbf{c}(\mathbf{z})$ represent the Lagrangian Gradient, Objective Gradient, Lagrange multipliers, and Constraint Jacobian, respectively. We then apply *ADiGator* to the file `basic_laggrad` using `z` as our variable of differentiation to create a file which is used to build the Lagrangian Hessian. In order to supply the Lagrangian Hessian to `fmincon`, we again write a wrapper which will call the generated derivative file `basic_laggrad_z` and build the Lagrangian Hessian using the required input/output scheme of `fmincon`. Here we note that, since we solve the problem at four different values of K, that this changes our problem size, and thus all derivative files must be generated prior to each call to `fmincon`.

Supplying Derivatives with Vectorization

Here we note that a part of our problem, namely, $\mathbf{F}(\mathbf{X},\mathbf{U})$, is of the vectorized nature in that $\mathbf{F}_i$ is only a function of $\mathbf{X}_i$ and $\mathbf{U}_i$ $(i=1,\ldots,N)$. As such, we may use *ADiGator* in the vectorized mode in order to differentiate this sub-problem. In order to do so, we first define a variable $\mathbf{y}(t)=[\mathbf{x}^T(t),u(t)]^T$, we then apply *ADiGator* in the vectorized mode to the file `F = dynamics(X,U)` with $\mathbf{y}(t)$ as our variable of differentiation. This essentially gives us a file which will compute $\mathbf{Jf}(\mathbf{y}(t))$ at the time points $t_1,\ldots,t_N$, given the inputs $\mathbf{X}$ and $\mathbf{U}$, where N may be any positive integer. In order to supply the Constraint Jacobian, we then write a file `Ceq = vect_cons(X,F,tf,probinfo)`, where, unlike in the non-vectorized case, this takes `F` as an input rather than calling the dynamics file. We then use the sparsity pattern of $\mathbf{Jf}(\mathbf{y}(t))$ together with the function `adigatorProjectVectLocs` to define the sparsity pattern of $\mathbf{JF}(\mathbf{z})$, given a fixed value of N, and apply *ADiGator* in the *non-vectorized* mode to the file `vect_cons` with $\mathbf{z}$ as our variable of differentiation. In order to supply the Constraint Jacobian to `fmincon`, we then write a wrapper file `vect_conswrap` which calls our vectorized dynamics derivative file, and uses the derivative outputs as inputs to the constraint derivative file and builds the Constraint Jacobian.

In order to supply the Lagrangian Hessian in this manner, we first must create a file which computes the second derivatives of the dynamics file, that is, $\bigtriangledown^2_{\mathbf{y}(t)}\mathbf{f}(t)$. This is done by applying *ADiGator* to our previously created vectorized dynamics derivative file, `dynamics_Y(X,U)`, again using $\mathbf{y}(t)$ as our variable of differentiation. Now, in order to get the Lagrangian Hessian, we first write a file which computes the Lagrangian Gradient, and take the derivative with respect to $\mathbf{z}$. In this example, we wrote this file as `GL = vect_laggrad(X,dX,F,dF,tf,dtf,probinfo)`, where `X`,`F`, `dF`, and `tf` are all identified as derivative inputs. Furthermore, the input `dF` is the non-zero derivatives of $\mathbf{JF}(\mathbf{z})$, and as such, identifying the non-zero locations of the derivative of `dF` with respect to $\mathbf{z}$ can be tricky. After identifying the derivative inputs properly, *ADiGator* is called in the non-vectorized mode on the file `vect_laggrad` using $\mathbf{z}$ as the variable of differentiation. In order to supply the Lagrangian Hessian we write the file `vect_laggradwrap` which first calls the second derivative of the dynamics file and then calls the derivative of the Lagrangian Gradient file. Here we note that since the derivatives of the dynamics file are done in the vectorized mode, that they only need to be generated once. Since the other derivative files are generated in the non-vectorized mode, they must be generated each time the value of K is changed. We also note that, for this example, one does not gain a lot of efficiency by taking advantage of the vectorized nature of the dynamics (primarily because the dynamics are fairly simple), but hopefully it will give the user some insight on how a problem may be separated into a vectorized part and a non-vectorized part.

12.3.3 Minimum Time to Climb of Supersonic Aircraft

In this example, we solve a discretized form of the continuous time optimal control problem:

$$\min_{\mathbf{x}(t),u(t),t_f} t_f \tag{23}$$

subject to the dynamic constraints

$$\mathbf{f}(\mathbf{x}(t),\mathbf{u}(t))=\begin{bmatrix}\dot{x}_1(t)\\ \dot{x}_2(t)\\ \dot{x}_3(t)\end{bmatrix}=\begin{bmatrix}x_2(t)\sin x_3(t)\\ \frac{\zeta(\mathbf{x}(t))-\theta(\mathbf{x}(t))}{c_2}-c_1\sin x_3(t)\\ \frac{c_1}{x_2(t)}(u(t)-\cos x_3(t)\end{bmatrix} \tag{24}$$

and boundary conditions

$$\begin{array}{ll}x_1(0)=b_1, & x_1(t_f)=b_2\\ x_2(0)=b_3, & x_2(t_f)=b_4\\ x_3(0)=b_5, & x_3(t_f)=b_5.\end{array} \tag{25}$$

As with the previous example, we use the Hermite Simpson Separated collocation method and as such we will be using the same notation. For this example, though, we rewrite the equality conditions of 20 into the form:

$$\mathbf{C}(\mathbf{X},\mathbf{U},t_N)=\mathbf{AX}+t_N\mathbf{BF}(\mathbf{X},\mathbf{U}), \tag{26}$$

where $\mathbf{C}(\mathbf{X},\mathbf{U},t_N)\in\mathbb{R}^{(N-1)\times 3}$. We also use the unrolled form:

$$\mathbf{C}^\dagger(\mathbf{X}^\dagger,\mathbf{U},t_N)=\mathbb{A}\mathbf{X}^\dagger+t_N\mathbb{B}\mathbf{F}^\dagger(\mathbf{X}^\dagger,\mathbf{U}). \tag{27}$$

As in the previous example, we define our NLP decision vector as $\mathbf{z} = [\mathbf{X}^{\dagger T}, \mathbf{U}^T, t_N]^T$. Our discretized problem is then

$$\min_{\mathbf{z}} t_N \tag{28}$$

subject to the equality constraints

$$\mathbf{C}^\dagger(\mathbf{z}) = \mathbf{0} \tag{29}$$

and the simple bounds

$$\mathbf{z}_{min} \leq \mathbf{z} \leq \mathbf{z}_{max}. \tag{30}$$

Now, in this example, we are concerned with building the Constraint Jacobian and the Lagrangian Hessian, which we will denote by $\bigtriangledown_{\mathbf{z}}\mathbf{C}^\dagger$ and $\bigtriangledown^2_{\mathbf{z}}L$, respectively. In building these, we take advantage of the fact that $\mathbf{F}^\dagger(\mathbf{X}^\dagger, \mathbf{U})$ is our only non-linear term, thus, *ADiGator* is only used to differentiate our dynamics file and we then build the Constraint Jacobian and Lagrangian Hessian using the result.

In order to build the Constraint Jacobian we first note that

$$\bigtriangledown_{\mathbf{z}}\mathbf{C}^\dagger = \begin{bmatrix} \bigtriangledown_{\mathbf{X}^\dagger}\mathbf{C}^\dagger & \bigtriangledown_{\mathbf{U}}\mathbf{C}^\dagger & \bigtriangledown_{t_N}\mathbf{C}^\dagger \end{bmatrix}, \tag{31}$$

where

$$\bigtriangledown_{\mathbf{X}^\dagger}\mathbf{C}^\dagger = \mathbb{A} + t_N\mathbb{B} \otimes \bigtriangledown_{\mathbf{X}^\dagger}\mathbf{F}^\dagger. \tag{32}$$

Here we note that we can take advantage of the way in which we write the unrolled Jacobian in order to perform the matrix multiplication $\mathbb{B} \otimes \bigtriangledown_{\mathbf{X}^\dagger}\mathbf{F}^\dagger$. That is, $\mathbb{B} \in \mathbb{R}^{((N-1)*3)\times(N*3)}$, and $\bigtriangledown_{\mathbf{X}^\dagger}\mathbf{F}^\dagger \in \mathbb{R}^{(N*3)\times(N*3)}$, thus we can perform the matrix multiplication by reshaping $\bigtriangledown_{\mathbf{X}^\dagger}\mathbf{F}^\dagger$ into a matrix of size $N \times (3 * N * 3)$, do the matrix multiplication, and then reshape the result into a matrix of size $((N-1)*3) \times (N*3)$. One should note, though, that this only works if multiplying by a matrix on the left side. Similarly, we can compute $\bigtriangledown_{\mathbf{U}}\mathbf{C}^\dagger$ and $\bigtriangledown_{t_N}\mathbf{C}^\dagger$ as

$$\bigtriangledown_{\mathbf{U}}\mathbf{C}^\dagger = t_n\mathbb{B} \otimes \bigtriangledown_{\mathbf{U}}\mathbf{F}^\dagger \tag{33}$$

and

$$\bigtriangledown_{t_N}\mathbf{C}^\dagger = \mathbb{B}\mathbf{F}^\dagger. \tag{34}$$

We can do similar calculations in order to build the Lagrangian Hessian. First we start with the Lagrangian

$$L = t_N + \boldsymbol{\lambda}^T\left(\mathbb{A}\mathbf{X}^\dagger + t_N\mathbb{B}\mathbf{F}^\dagger\right). \tag{35}$$

For these calculations, we first define a new variable $\mathbf{Y} = [\mathbf{X}, \mathbf{U}]$, now, using some abusive notation, we may write our Lagrangian Gradient as

$$\bigtriangledown_{\mathbf{z}}L = \begin{bmatrix} \bigtriangledown_{\mathbf{Y}^\dagger}L & \bigtriangledown_{t_N}L \end{bmatrix} \tag{36}$$

where

$$\bigtriangledown_{\mathbf{Y}^\dagger}L = \boldsymbol{\lambda}^T\left(\mathbb{A} \otimes [\mathbf{1}, \mathbf{0}] + \mathbf{t_N}\mathbb{B} \otimes \bigtriangledown_{\mathbf{Y}^\dagger}\mathbf{F}^\dagger\right) \tag{37}$$

and

$$\bigtriangledown_{t_N}L = \boldsymbol{\lambda}^T\left([\mathbf{0}, \mathbf{1}]^{\mathbf{T}} + \mathbb{B}\mathbf{F}^\dagger\right). \tag{38}$$

Now, we write our Lagrangian Hessian as

$$\bigtriangledown^2_{\mathbf{z}}L = \begin{bmatrix} \bigtriangledown^2_{\mathbf{Y}^\dagger}L & \bigtriangledown_{\mathbf{Y}^\dagger t_N}L \\ \bigtriangledown_{\mathbf{Y}^\dagger t_N}L & \bigtriangledown^2_{t_N}L \end{bmatrix}, \tag{39}$$

where,

$$\bigtriangledown^2_{\mathbf{Y}^\dagger}L = t_N\boldsymbol{\lambda}^T\mathbb{B} \otimes \bigtriangledown^2_{\mathbf{Y}^\dagger}\mathbf{F}^\dagger, \tag{40}$$

$$\bigtriangledown_{\mathbf{Y}^\dagger t_N}L = \boldsymbol{\lambda}^T\mathbb{B}\,\bigtriangledown_{\mathbf{Y}^\dagger}\mathbf{F}^\dagger, \tag{41}$$

and

$$\bigtriangledown^2_{t_N}L = 0. \tag{42}$$

The code for this example may be found in the directory `examples/optimization/vectorized/minimumclimb`. In this directory there are four files which will solve the problem on three different meshes ($K = 10$, $K = 20$, and $K = 40$), where the initial guess for the second two iterations is calculated from the solution of

the previous mesh. The files `main_1stderivs_nonvect` and `main_2ndderivs_nonvect` supply derivatives without using the vectorized mode, while the files `main_1stderivs_vect` and `main_2ndderivs_vect` supply derivatives using the vectorized mode. Furthermore, the function which computes the Constraint Jacobian (and calls the first dynamics derivative file) is given in `conswrap.m` and the function which computes the Lagrangian Hessian (and calls the second dynamics derivative file) is given in `laghesswrap.m`.

Here we see that, since we are only differentiating the dynamics file, that, when we use the vectorized mode, we must only create the derivative files a single time and they may be used to solve on as many different meshes as desired. If using the non-vectorized mode, however, the derivative files must be generated each time the value of K is changed. As far as performance goes, the user should note a slight increase in performance using the vectorized mode at the first derivative (the vectorized file should solve in 90-95% of the time it takes the non-vectorized). At the second derivative level, though, we see a major increase in performance as the vectorized file should solve in 5-10% of the time it takes the non-vectorized.

12.4 Extended-Feature and Embedded Examples (v2.0)

The following examples exercise the fork-specific capabilities of Section 6. Each is self-checking — it compares the generated derivative against a closed form and/or central finite differences and prints a one-line report.

12.4.1 Reverse-Mode Gradient of Log-Sum-Exp

The code in `examples/gradients/logsumexp/` takes the gradient of the scalar cost $\log \sum_i e^{w_i x_i}$ with respect to `x` using the reverse-mode routine `adigatorGenRevGradFile` (Section 5.8). The generated `lse_cost_RGrd` returns the full n-vector gradient and the value from a single forward and reverse sweep, independent of n:

```
gx = adigatorCreateDerivInput([n 1],'x');
gw = adigatorCreateAuxInput([n 1]);
adigatorGenRevGradFile('lse_cost',{gx,gw},adigatorOptions('overwrite',1));
[g,y] = lse_cost_RGrd(x,w);   % output order: [Grd, Fun]
```

The driver checks the gradient against the closed form $w_i e^{w_i x_i} / \sum_j e^{w_j x_j}$ and against central differences. The differentiated cost function `lse_cost.m` is:

```
function y = lse_cost(x,w)
% Weighted log-sum-exp scalar cost (roadmap R4 reverse-mode example):
% a dense-gradient objective where forward mode costs O(n) passes and the
% reverse (adjoint) sweep costs O(1) function evaluations.
y = log(sum(exp(w.*x)));
end
```

12.4.2 Runtime Loop Bounds

The code in `examples/jacobians/loopbound/` demonstrates the `LOOPBOUND` option (Section 6.4): a reduction over `N` actuators is differentiated once at a maximum trip count `Nmax`, and the single generated file is then evaluated for several `n` $\leq$ `Nmax` without regeneration, with exact structural zeros beyond `n`. This is the "reconfigurable number of actuators" use case; the driver confirms the gradient matches the directly computed `n`-sized problem and that the padded tail stays zero. The differentiated function `lb_alloc.m` is:

```
function [J,v] = lb_alloc(x,p,N)
% Per-actuator cost terms accumulated in a rolled loop with a RUNTIME
% bound N (roadmap R3; issue #6 Tier 1). Generated once with N = Nmax,
% the derivative file accepts any 1 <= N <= Nmax at runtime: v is
% allocated as zeros(N,1) at the runtime value (the bound parameter
% prints by name), skipped iterations contribute nothing to the
% accumulator J, and the derivative pattern keeps the Nmax shape with
% structural zeros beyond N (padded-program semantics).
%
% Embeddability. This function alone does not code-generate under static memory
% allocation: v = zeros(N,1) has no compile-time bound when N is a runtime
% input (measured). The generated derivative does, because 'loopbound' declares
% Nmax and emits the runtime guard that lets Coder size everything statically.
```

```
% So a runtime bound is not itself a barrier to an embeddable derivative -- an
% UNDECLARED one is. Contrast examples/stiffodes/, where the same runtime-sized
% shape has no loop for 'loopbound' to match and the size must be pinned.
v = zeros(N,1);
J = 0;
for a = 1:N
  ua   = x(a);
  v(a) = p(a,1)*ua^2 + p(a,2)*ua;
  J    = J + v(a);
end
end
```

12.4.3 N-Dimensional Parameters

The code in `examples/jacobians/ndparam/` declares a parameter `B` of size $m \times n \times K$ with `adigatorCreateAuxInput([m n K])` and slices it inside the differentiated function with the natural `B(:,:,k)` syntax (Section 6.2). The generated file accepts `B` either as the 3-D array or as its 2-D column-major fold; the driver checks the Jacobian against the analytic result and finite differences. The differentiated function `tvmap.m` is:

```
function y = tvmap(x,B)
% Time-varying linear map (roadmap R2; issue #11 Level 2 veneer).
%
% B is an N-D declared parameter of size m x n x K (see
% adigatorCreateAuxInput): inside the rolled loop it is sliced with the
% natural B(:,:,k) syntax, k being the loop counter. ADiGator rewrites the
% slice as the affine column window (k-1)*n + (1:n) on the internal 2D
% fold of B.
%
% y stacks y_k = B(:,:,k)*g(x), g(x) = x + x.^3/6, over k = 1..K.
K = 4;
m = 3;
g = x + x.^3/6;
y = zeros(m*K,1);
for k = 1:K
  y((k-1)*m+(1:m)) = B(:,:,k)*g;
end
end
```

12.4.4 Struct Inputs

The code in `examples/jacobians/structinput/` carries the variable of differentiation and the auxiliary data as fields of a scalar struct (Section 6.3), exercising four cases: a flat-struct Hessian, a nested-struct Hessian (the derivative field one level deeper), a vector-function Jacobian, and the same flat-struct Hessian through the embedded inline (`'i'`) pipeline. Each case is checked against the closed-form gradient and Hessian of $\frac{1}{2}\mathbf{x}^\top\mathbf{Q}\mathbf{x} + \mathbf{c}^\top\mathbf{x}$. The flat-struct objective `structobj.m` is:

```
function f = structobj(in)
% Example user function whose inputs are carried as fields of a single
% struct (issue #24, scope A):
%   in.x : optimization variable, differentiated with respect to
%   in.Q : symmetric weight matrix (auxiliary, no derivatives)
%   in.c : linear term            (auxiliary, no derivatives)
%
% Scalar quadratic objective f(x) = 1/2 x'Q x + c'x, so that
%   grad_x f = Q x + c   and   hess_x f = Q,
% giving closed-form references for the example's checks.
%
% Copyright Pedro Lourenço and GMV. Distributed under the GNU General Public License v3.0.
f = 0.5*(in.x.'*(in.Q*in.x)) + in.c.'*in.x;
end
```

12.4.5 Allocation over Time (Free Dimensions)

The code in `examples/optimization/vectorized/allocation/` differentiates the per-actuator, per-time-step terms of an optimal-allocation problem *once* in vectorized mode over the product index $i = (k-1)N + a$, leaving both the actuator count `N` and the horizon `K` free at runtime. A single generated derivative file then serves every (N, K); the reductions (objective sum, per-time-step moments) are assembled with plain linear algebra by `alloc_assemble.m`. See the folder's `README.md` for the full pattern catalog (product fold, assembly wrappers, folded-2-D parameters, cell arrays).

12.4.6 Embedded Hessian: PIPG Gap Function

The code in `examples/optimization/pipg/` generates an embeddable Hessian of a small cone/gap function, $f = \mathbf{w}^\top \texttt{conefun}(\mathbf{z}) + \mathbf{z}^\top \texttt{setfun}(\mathbf{z})$, with `adigatorGenDerFile_embedded` (Section 6.1), producing the wrapper triple `[H,G,f] = gapfun_Hes(w,z)`.

12.5 Other

12.5.1 Jacobian-Vector and Hessian-Vector Products

This code is contained within `examples/jachesvecprods/`. It shows the user how to generate functions which automatically compute Jacobian-vector products and Hessian-vector products via seeding. Note: If a Jacobian-vector product is required, it is likely most efficient to generate the file as a Jacobian-vector product. If a Hessian-vector product is required, it is more than likely more efficient to compute the Hessian and perform the product for "small" problem sizes ($n < 10000$). As n increases beyond 10000 or so, the Hessian-vector product via seeding is likely the better option.

12.5.2 Hessian of the Log-Sum-Exp Function

This code is contained within `examples/hessians/logsumexp/`. It simply uses the `adigatorGenHesFile` routine to generate a Hessian file wrapper.

12.5.3 Unconstrained Brown Minimization

This code is contained within `examples/optimization/fminuncEx/`. The code uses `adigatorGenFiles4Fminunc` together with `fminunc` to minimize the brown function. *Note: this example demonstrates a deprecated wrapper (Section 5.3); it is retained for upstream continuity and is not embeddable.*

12.5.4 Banana System of Non-Linear Equations

This code is contained within `examples/optimization/fsolveEx/`. The code uses the `adigatorGenFiles4Fsolve` together with `fsolve` to solve the banana system of equations. *Note: this example demonstrates a deprecated wrapper (Section 5.3); it is retained for upstream continuity and is not embeddable.*

12.5.5 Minimization Ginzburg-Landau (2-dimensional) Superconductivity Problem

This code is contained within `examples/optimization/ipoptEx/`. The code uses `adigatorGenFiles4Ipopt` together with `ipopt` in order to minimize the MINPACK-2 GL2 minimization problem. *Note: this example demonstrates a deprecated wrapper (Section 5.3); it is retained for upstream continuity and is not embeddable.*

Appendix

In this user guide we employ the following notation. First, all scalars, vectors, and matrices are denoted by lower or upper case non-boldface (e.g., y or Y), lower case boldface (e.g., $\mathbf{y}$), and upper case boldface (e.g, $\mathbf{Y}$)

symbols, respectively. Furthermore, if we are referring to a variable in MATLAB, we will generally use the typewriter text (e.g., `y, Y`). Thus, if $\mathbf{X} \in \mathbb{R}^{m\times n}$, then

$$\mathbf{X} = \begin{bmatrix} X_{1,1} & \cdots & X_{1,n} \\ X_{2,1} & \cdots & X_{2,n} \\ \vdots & \ddots & \vdots \\ X_{m,1} & \cdots & X_{m,n} \end{bmatrix} \in \mathbb{R}^{m\times n}, \tag{43}$$

where $X_{i,j},\ (i = 1,\ldots,m),\ (j = 1,\ldots,n)$ are the *elements* of the $m \times n$ matrix $\mathbf{X}$. Similarly, the output of any matrix function of a matrix is denoted by a upper case bold letter. Consequently, if $\mathbf{F} : \mathbb{R}^{m\times n} \longrightarrow \mathbb{R}^{p\times q}$ is a matrix function of the matrix variable $\mathbf{X}$, then $\mathbf{F}(\mathbf{X})$ has the form

$$\mathbf{F}(\mathbf{X}) = \begin{bmatrix} F_{1,1}(\mathbf{X}) & \cdots & F_{1,q}(\mathbf{X}) \\ F_{2,1}(\mathbf{X}) & \cdots & F_{2,q}(\mathbf{X}) \\ \vdots & \ddots & \vdots \\ F_{p,1}(\mathbf{X}) & \cdots & F_{p,q}(\mathbf{X}) \end{bmatrix} \in \mathbb{R}^{p\times q}, \tag{44}$$

where $F_{k,l}(\mathbf{X}),\ (k = 1,\ldots,p),\ (l = 1,\ldots,q)$ are the *elements* of the $p \times q$ matrix function $\mathbf{F}(\mathbf{X})$. The *Jacobian* of the matrix function $\mathbf{F}(\mathbf{X})$, denoted $\mathbf{JF}(\mathbf{X})$, is then a four-dimensional array of size $p \times q \times m \times n$ that consists of $pqmn$ elements. This multi-dimensional array will be referred to generically as the *rolled* representation of the derivative of $\mathbf{F}(\mathbf{X})$ with respect to $\mathbf{X}$ (where the term "rolled" is similar to the term "external" as used in [9]). In order to provide a more tractable form for the Jacobian of $\mathbf{F}(\mathbf{X})$, the matrix variable $\mathbf{X}$ and matrix function $\mathbf{F}(\mathbf{X})$ are transformed into the following so called *unrolled* form (where, again, the term "unrolled" is similar to the term "internal" [9]). First, $\mathbf{X} \in \mathbb{R}^{m\times n}$, is mapped isomorphically to a column vector $\mathbf{x}^\dagger \in \mathbb{R}^{mn}$, such that

$$x^\dagger_k = X_{i,j}, \quad \text{where} \quad k = i + m(j-1), \quad \forall \quad \begin{array}{rcl} i & = & 1,\ldots m \\ j & = & 1,\ldots n \\ k & = & 1,\ldots mn \end{array} . \tag{45}$$

Similarly, let $\mathbf{f}^\dagger(\mathbf{x}^\dagger) \in \mathbb{R}^{pq}$ be the one-dimensional transformation of the function $\mathbf{F}(\mathbf{X}) \in \mathbb{R}^{p\times q}$, such that

$$f^\dagger_k(\mathbf{x}^\dagger) = F_{i,j}(\mathbf{X}), \quad \text{where} \quad k = i + q(j-1), \quad \forall \quad \begin{array}{rcl} i & = & 1,\ldots q \\ j & = & 1,\ldots p \\ k & = & 1,\ldots qp \end{array} . \tag{46}$$

Here we note that the transformation from $\mathbf{X}$ to $\mathbf{x}^\dagger$ can be performed in MATLAB using the command `xdag = X(:)`, where `xdag` and `X` correspond to $\mathbf{x}^\dagger$ and $\mathbf{X}$, respectively. Furthermore, the MATLAB functions `sub2ind` and `ind2sub` are useful when switching between rolled and unrolled representations.

Using the one-dimensional representations $\mathbf{x}^\dagger$ and $\mathbf{f}^\dagger(\mathbf{x}^\dagger)$, the four-dimensional Jacobian, $\mathbf{JF}(\mathbf{X})$ can be represented in two-dimensional form as

$$\mathbf{Jf}^\dagger(\mathbf{x}^\dagger) = \begin{bmatrix} \frac{\partial f^\dagger_1}{\partial x^\dagger_1} & \cdots & \frac{\partial f^\dagger_1}{\partial x^\dagger_{mn}} \\ \frac{\partial f^\dagger_2}{\partial x^\dagger_1} & \cdots & \frac{\partial f^\dagger_2}{\partial x^\dagger_{mn}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f^\dagger_{pq}}{\partial x^\dagger_1} & \cdots & \frac{\partial f^\dagger_{pq}}{\partial x^\dagger_{mn}} \end{bmatrix} \in \mathbb{R}^{pq\times mn}. \tag{47}$$

Equation (47) provides what we refer to as the "unrolled Jacobian".

References

[1] M. J. Weinstein and A. V. Rao. A source transformation via operator overloading method for automatic differentiation in matlab. *ACM Transactions on Mathematical Software*, In Revision 2014.

[2] M. A. Patterson, M. J. Weinstein, and A. V. Rao. An efficient overloaded method for computing derivatives of mathematical functions in matlab. *ACM Transactions on Mathematical Software,*, 39(3):17:1–17:36, July 2013.

[3] A. Griewank. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation. Frontiers in Appl. Mathematics*. SIAM Press, Philadelphia, Pennsylvania, 2008.

[4] L. T. Biegler and V. M. Zavala. Large-scale nonlinear programming using IPOPT: An integrating framework for enterprise-wide optimization. *Computers and Chemical Engineering*, 33(3):575–582, March 2008.

[5] A. Waechter and L. T. Biegler. On the implementation of a primal-dual interior-point filter line search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):575–582, March 2006.

[6] Christian H. Bischof, H. Martin Bücker, Bruno Lang, A. Rasch, and Andre Vehreschild. Combining source transformation and operator overloading techniques to compute derivatives for MATLAB programs. In *Proceedings of the Second IEEE International Workshop on Source Code Analysis and Manipulation (SCAM 2002)*, pages 65–72, Los Alamitos, CA, USA, 2002. IEEE Computer Society.

[7] G Wanner and E Hairer. *Solving Ordinary Differential Equations II*, volume 1. Springer-Verlag, Berlin, 1991.

[8] T. Burg, D. Dawson, and P. Vedagarbha. A redesigned dcal controller without velocity measurements: theory and demonstration. *Robotica*, 15:337–346, 5 1997.

[9] S. A. Forth. An efficient overloaded implementation of forward mode automatic differentiation in MATLAB. *ACM Transactions on Mathematical Software*, 32(2):195–222, April–June 2006.